



BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI

MTech. In Software Systems

PETURA: Animal Welfare System

Submitted by

Manmeet Chhabra

2020WA15293

Dissertation Work carried out at Wipro Technologies, Bangalore

SSWTZG628T DISSERTATION

PETURA: Animal Welfare System

Outline of Dissertation submitted in partial

Fulfillment of the requirements of

M. Tech. Software Systems Degree Program

By

Manmeet Chhabra (2020WA15293)

Under the supervision of

Raj Kumar

Dissertation work carried out at

Wipro Technologies, Bangalore

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE

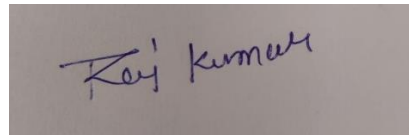
PILANI (RAJASTHAN)

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

CERTIFICATE

This is to certify that the Dissertation entitled **PETURA: Animal Welfare System** submitted by Manmeet Chhabra ID No. 2020WA15293.

In partial fulfillment of the requirements of SSWTZG628T Dissertation, embodies the work done by her under my supervision.

A rectangular box containing a handwritten signature in dark ink. The signature appears to be 'Raj Kumar' written in a cursive style.

Signature of the Supervisor
Name: Raj Kumar
Designation: Project Lead

Date: 10/11/2024

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

First Semester 2024- 25

SSWT ZG628T DISSERTATION

FINAL SEMESTER EVALUATION FORM

Section I

(To be filled by the Student and returned to the Supervisor)

ID No.: 2020WA15293

Name of Student: Manmeet Chhabra

Name of Supervisor: Raj Kumar

Name of the Examiner(s): Aman Dixit & Perna Kumari

Dissertation Title: “PETURA: Animal Welfare System”

Section II

(To be filled by the Supervisor in consultation with the examiner(s))

Comments on the dissertation from Examiner and Supervisor (Select Y or N)

1. Quantum of work

- | | |
|---|---|
| a. Justifiable as efforts for 5 weeks duration | Y |
| b. Work is in line with the commitments made in outline | Y |

2. Type of work

- | | |
|--|---|
| a. Client assignment | N |
| b. Organization specific task | N |
| c. General study project such as white paper | Y |
| d. Any other (kindly elaborate below in a line or two if Y | N |

3. Nature of work

- | | |
|--|---|
| a. Routine in nature | Y |
| b. Involved creativity and rational thinking | Y |

Kindly elaborate below if answer for above is "Y":

The PETURA animal welfare system successfully meets its objectives, delivering a comprehensive, secure, and scalable platform. It demonstrates strong functionality and user engagement.

4. Evaluation methodology

- | | |
|---|---|
| a. Evaluation done based on presentation to supervisor and examiner | Y |
| b. Evaluation done through Viva conducted by supervisor and examiner | Y |
| c. Student regularly interacted with supervisor and incorporated the suggestions made | Y |
| d. Brief description on the report submitted, quality of presentation and suggestions given for improvement | |

The report submitted is informative and properly describing the understanding of the work. Documentation for each module is precise and telling the true nature of the project's functionalities. Presentation given by Manmeet was detailed and accurate.

5. Final semester evaluation matrix

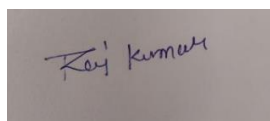
Tick the appropriate box (1 is lowest and 5 is the highest)

Dimension	Rank→	1	2	3	4	5
Student abilities in general						
Understanding of the subject of dissertation						5
Creative thinking ability to come up with new ideas						5
Viva / Seminar presentation						
Communication ability						5
Organization of material					4	
Response to review questions						5
Cohesive thinking ability					4	
Report submitted						
Report structure and format						5
Technical content of the report						5
Explanation on the significance of the assignment						5
Analysis of alternative approaches					4	

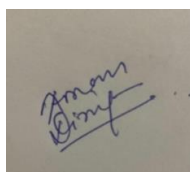
Any other comments:

The application exhibits a high level of functionality and attention to detail, effectively solving real-world challenges in animal welfare management. The modules are well-integrated, and the documentation offers clear guidance on how each part contributes to the overall system. Manmeet's presentation demonstrated a strong grasp of the project, highlighting its potential for scalability and further development.

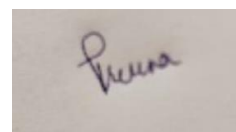
Date: 09/09/2024



Signature of Supervisor



Signature of Examiner



Signature of Examiner

Acknowledgement

I would like to thank my supervisor Raj Kumar for his guidance, time, and patience. He has been extremely supportive and co-operative in this whole effort. His valuable comments and support have been immensely helpful in enhancing the quality of work.

Special thanks to my examiners Aman Dixit and Prerna Kumari for their wholehearted support and timely guidance.

I'm also thankful to WIPRO Technologies for giving me this opportunity and making available all the resources required for this dissertation.

I'm also thankful to BITS, Pilani, my entire supporting faculty and the training department for all their efforts in organizing this course.

Manmeet Chhabra

2020WA15293

Table of Content

CHAPTER NO	TITLE	PAGE NUMBER
	Abstract	11
	LIST OF ABBREVIATIONS USED	Dec-13
	LIST OF FIGURES AND SCREENSHOTS	14
1	INTRODUCTION	15-17
	1.1 Overview of PETURA	15
	1.2 Problem Statement	15
	1.3 Objective	15-16
	1.4 Scope	16
	1.5 Limitation	16
	1.6 Key Features and Functionalities	16
	1.7 Summary of Methodology.	17
	1.8 Target User and Stakeholders	17
2	LITERATURE REVIEW	18-19
	2.1 Background on Animal Welfare Systems	18
	2.2 Existing Solutions and Technologies	18
	2.3 Challenges in Animal Welfare System	19
3	FEASIBILITY STUDY	20-21
	3.1 Technical Feasibility	20
	3.2 Economical Feasibility	20
	3.3 Operational Feasibility	21
	3.4 Legal and Ethical Consideration	21
4	TOOLS AND TECHNOLOGIES	22-30
	4.1 Tools	22-23
	4.2 Technologies	24-30
	4.2.1 Frameworks	24-25
	4.2.2 Database	25-26
	4.2.3 Frontend	26-29
	4.2.4 Libraries Used	29-30
	4.2.5 External Tools	30
5	SYSTEM REQUIREMENTS	31-32
	5.1 Hardware Requirements	31
	5.2 Software Requirements	31
	5.3 Development Tools	31-32
	5.4 External Libraries	32

6	SOFTWARE REQUIREMENT SPECIFICATION (SRS)	33-35
	6.1 Functional Requirements	33
	6.2 Non-Functional Requirements	34-35
7	SYSTEM ARCHITECTURE	36-39
	7.1 Overview of the Architecture	36
	7.2 High Level Diagram	36-37
	7.3 Component Design	37-38
	7.4 Data Flow	38-39
8	DETAILED DESIGN	40-54
	8.1 Flow Chart	40-47
	8.2 Use Case Diagram	48
	8.3 Data Flow Diagram	49-50
	8.4 Entity Relationship Diagram (ERD)	51
	8.5 Database Schema	52-54
9	MODULES IN PETURA	55-59
	9.1 User Module	55
	9.2 Animal Module	55
	9.3 Shelter Module	56
	9.4 Adoption Module	56
	9.5 Incident Report Module	57
	9.6 Donation Module	57
	9.7 Report Generation Module	58
	9.8 Notification Module	58
	9.9 Search and Filter Module	59
10	SECURITY FEATURES	60-63
	10.1 Authentication	60
	10.2 Authorization	61
	10.3 Data Protection	62
11	USER ROLES AND PERMISSIONS	64-67
	11.1 Overview of Spring Security Configuration	64
	11.2 Role-Based Dashboard	65
	11.3 Admin Role	66
	11.4 Staff Role	66
	11.5 User Role	66
	11.6 Permission Table	67
12	SYSTEM WORKFLOW	68-70
	12.1 Adoption Process	68

	12.2 Donation Process	68-69
	12.3 Incident Reporting Process	69
	12.4 Animal Management Flow	70
13	USER INTERFACE	71-74
	13.1 Dashboard	71
	13.2 Pages and Forms	72
	13.3 Responsive Design Approach	73
14	REPORTS AND ANALYTICS	75-78
	14.1 Custom Report Generation (Excel & PDF)	75-78
15	MAP INTEGRATION	79-83
	15.1 Overview of Map Integration (Leaflet)	79
	15.2 Technical Implementation	79-82
	15.3 User Interaction with Map	82
16	EMAIL NOTIFICATION AND ALERTS	84-87
	16.1 Registration Notifications	84
	16.2 Donation Acknowledgements	85
	16.3 Adoption Status Tracker	85-86
	16.4 Incident Alerts	86-87
17	COMPREHENSIVE VIEW OF APPLICATION	88-101
	17.1 User Module	88-90
	17.2 Animal Module	90-92
	17.3 Shelter Module	92-93
	17.4 Adoption Module	93-95
	17.5 Donation Module	95-98
	17.6 Report Generation Module	98-100
	17.7 Incident Report Module	100-101
18	CHALLENGES FACED	102
	19.1 Data Validation	102
	19.2 Notification Management	102
	19.3 Performance Scalability	102
	19.4 Security Concerns	102
19	FUTURE ENHANCEMENTS	103
20	CONCLUSION	104
21	REFERENCES	105
22	APPENDICES	106-109
23	DETAILED PLAN OF WORK	110

ABSTRACT

"PETURA: Love Knows No Species" is a comprehensive animal welfare system designed to streamline and enhance animal management, adoption, and support for shelters and pet enthusiasts. The application integrates multiple modules, including user management, animal and shelter information, adoption tracking, incident reporting, donation management, and a notification system.

The primary objective of PETURA is to create a centralized, user-friendly platform that facilitates efficient operations within animal shelters and encourages public engagement in animal welfare efforts. Through role-based access, the system caters to various users, including administrators, shelter staff, and general users. Key features include a searchable database for animals, detailed reporting tools with custom report generation, and map integration to enhance the accessibility of shelter locations and reported incidents.

Built using Spring Boot and Thymeleaf with SQL for data management, PETURA ensures a secure and responsive experience, meeting technical, operational, and ethical requirements of an animal welfare ecosystem. The system's structure supports modular scalability and future enhancements, addressing common challenges in animal welfare with a focus on transparency, community engagement, and data-driven decision-making.

This document details PETURA's development journey, from initial feasibility studies and literature review to system workflows, security configurations, and future enhancements. PETURA aims to support shelters and animal lovers alike, uniting them in the mission to protect and care for animals in need.

ABBREVIATIONS USED

Abbreviation	Full Form
API	Application Programming Interface
SQL	Structured Query Language
GDPR	General Data Protection Regulation
STS	Spring Tool Suite
IDE	Integrated Development Environment
HTTP	Hypertext Transfer Protocol
JPA	Java Persistence API
CLI	Command-Line Interface
ORM	Object-Relational Mapping
JPQL	Java Persistence Query Language
JWT	JSON Web Token
CSRF	Cross-Site Request Forgery
XSS	Cross-Site Scripting
RDBMS	Relational Database Management System
HTML	Hypertext Markup Language
XML	Extensible Markup Language
CSS	Cascading Style Sheets
WWW	World Wide Web
SMTP	Simple Mail Transfer Protocol
IMAP	Internet Message Access Protocol
POP3	Post Office Protocol version 3
JSON	JavaScript Object Notation
PDF	Portable Document Format
RAM	Random Access Memory
GB	Gigabyte

OS	Operating System
JDK	Java Development Kit
SSL/TLS	Secure Sockets Layer / Transport Layer Security
SRS	Software Requirements Specification
MVC	Model-View-Controller
DFD	Data Flow Diagram
ERD	Entity-Relationship Diagram
HTTPS	Hypertext Transfer Protocol Secure
UI	User Interface

FIGURES AND IMAGES USED

Figure	Page No.
Logo of PETURA	15
High Level Architecture	36
Flowcharts	40-47
Use Case Diagram	48
Data Flow Diagram	49-50
Entity Relationship Diagram	51
Database Schema	52-54
User Dashboard	72
Incident Report Form	73
Excel Report	76
PDF Report	77
Report Generation	78
Map Integration	80-82
User Registration Notification	84
Donation Notification	85
Adoption Notification	86
Incident Alerts	87
Output Screens	88-101
Appendix	106-109

Chapter-1: INTRODUCTION

1.1 Overview of PETURA

PETURA: Love Knows No Species is an innovative animal welfare management system that addresses the diverse needs of animal shelters, adopters, and the general public.



Fig 1.1 Logo of PETURA

Designed as a web application, PETURA combines animal management, user engagement, and educational resources in a cohesive and easy-to-use platform. Its primary functions include managing animal profiles, supporting adoption processes, handling incident reporting, enabling donations, and facilitating user notifications. PETURA aims to be an inclusive tool that assists shelters and encourages community involvement in animal welfare.

The PETURA application is an open-source project aimed at supporting animal shelters and welfare organizations. The source code is available on GitHub: [\[GitHub Link\]](#).

1.2 Problem Statement

Managing animal welfare involves complex and labor-intensive tasks, from tracking animal health and adoption to coordinating shelter operations and engaging with the public. Traditional methods often lack efficiency, transparency, and centralization, making it difficult for shelters to handle a high volume of data and interactions. PETURA addresses these gaps by providing a digital solution that integrates all aspects of animal shelter operations, thus reducing workload, enhancing accessibility, and promoting more effective animal care and public engagement.

1.3 Objective

The objective of PETURA is to create a comprehensive, user-friendly platform that simplifies the processes involved in animal welfare management. It aims to:

- Provide a structured system for managing animal profiles and adoption requests.

- Facilitate user engagement through role-based access, notifications, and educational resources.
- Support shelters by streamlining donations, incident reporting, and reporting tools.
- Promote transparency and data-driven decision-making in animal welfare.

1.4 Scope

The PETURA system covers several functional areas, including:

- **Animal Management:** Creation, update, and maintenance of animal profiles.
- **User Roles and Permissions:** Access control for administrators, staff, and general users.
- **Adoption Process:** End-to-end management of adoption requests and approvals.
- **Incident Reporting:** A module for reporting and tracking incidents.
- **Donation Management:** Enabling users to contribute one-time or recurring donations.
- **Notification System:** Alerts and notifications for registration, adoption status, and incidents.
- **Educational Resources:** Pages dedicated to animal care, grooming, and welfare tips.

1.5 Limitations

Mobile application development will not be covered under the project as of now.

Web-based interface and functionality only.

1.6 Key Features and Functionalities

PETURA includes the following main features:

- **Role-Based Dashboards:** Customized views and options based on user roles.
- **Comprehensive Search and Filter:** Enables users to search for animals based on specific criteria.
- **Incident Reporting with Map Integration:** Allows users to report incidents with location data.
- **Donation System:** Facilitates one-time and recurring donations with acknowledgment emails.
- **Automated Email Notifications:** System-generated emails for registration, donation, and adoption.
- **Report Generation:** Customizable reports for data analysis, available in Excel and PDF formats.
- **Educational Pages:** Sections on animal care, training, and welfare to engage users and improve awareness.

1.7 Summary of Methodology

The PETURA project follows a structured development methodology, involving:

- **Feasibility Study:** Evaluating technical, economic, and operational feasibility.
- **Requirement Analysis:** Gathering functional and non-functional requirements.
- **System Design:** Crafting system architecture and database schema.
- **Implementation:** Using Spring Boot and Thymeleaf to build backend and frontend components.
- **Testing and Validation:** Conducting rigorous testing to ensure functionality and security.
- **Documentation:** Creating detailed user guides, technical documentation, and support materials.

1.8 Target Users and Stakeholders

The primary users and stakeholders of PETURA include:

- **Administrators:** Responsible for managing system, user permissions, and high-level operations.
- **Shelter Staff:** Manages daily shelter activities, including animal and adoption management.
- **General Users:** Includes potential adopters, donors, and animal welfare enthusiasts.
- **Community and Volunteers:** People interested in supporting the mission through donations and volunteering.

Chapter-2: LITERATURE REVIEW

2.1 Background on Animal Welfare Systems

Animal welfare systems are designed to ensure the well-being and protection of animals, focusing on aspects such as humane treatment, ethical standards, and public awareness. Historically, animal welfare efforts were led by charitable organizations and local communities, with limited digital intervention. As the importance of animal rights and welfare has grown, there has been a shift towards integrating technology into animal shelter operations, adoption management, incident reporting, and community engagement. Digital systems have enhanced transparency and accountability, allowing shelters to efficiently manage resources, track animal information, and interact with the public. PETURA aims to contribute to this evolving field by providing a comprehensive platform tailored for shelters, animal enthusiasts, and the general public.

2.2 Existing Solutions and Technologies

Numerous digital solutions address aspects of animal welfare, ranging from standalone apps for pet adoption to sophisticated shelter management systems. Key existing solutions include:

- **Petfinder**: An online pet adoption platform that connects potential adopters with pets from various shelters and rescues. Petfinder allows users to search based on location, animal type, and adoption requirements, but does not provide broader shelter management features.
- **Shelterluv**: A cloud-based shelter management software that facilitates animal intake, adoption, and donations. Shelterluv integrates well with external tools but may lack the customization needed for smaller, unique shelters.
- **Petpoint**: A data-driven management solution offering a wide range of tools for shelter operations, including animal tracking, medical history, and adoption workflow. While robust, it can be costly for smaller organizations.
- **Adopt-a-Pet**: A platform that collaborates with shelters to list adoptable pets online and promote adoption events. It primarily serves as a search and listing tool without in-depth shelter management capabilities.

Technologies used in these solutions often include web-based applications with cloud storage for data accessibility, frontend frameworks like React or Angular for user interfaces, and backend systems that handle animal databases, adoption workflows, and donor management. Despite these advancements, challenges such as interoperability, cost, and customization remain, especially for smaller shelters.

2.3 Challenges in Animal Welfare Systems

Animal welfare systems face several challenges that PETURA aims to address:

- **Data Management and Accessibility:** Shelters often handle large volumes of data, including animal profiles, medical history, and adoption records, which can become overwhelming without an organized digital structure.
- **Adoption Process Complexity:** The adoption process includes eligibility screening, follow-up, and approval, often leading to delays. Current solutions sometimes lack automated workflows that streamline adoption tasks.
- **Community Engagement:** Many systems lack effective ways to engage the public beyond adoption listings. Integrating educational resources and communication tools is crucial to foster a more active animal welfare community.
- **Resource Constraints:** Smaller shelters have limited resources, which affects their ability to invest in costly or complex management systems. PETURA aims to be a cost-effective and modular solution suitable for shelters of all sizes.
- **Incident Reporting and Location Mapping:** Incident reports are essential for tracking animal-related cases, but few systems incorporate mapping functionalities that could enable users to report locations effectively. PETURA's integration of map-based incident reporting can address this gap.

Chapter-3: FEASIBILITY STUDY

3.1 Technical Feasibility

The technical feasibility of PETURA has been assessed based on the system's required tools, technology stack, and scalability:

- **Technology Stack:** PETURA is built on Spring Boot for the backend, Thymeleaf for the frontend, and SQL for the database, making it a robust and widely supported stack. This combination allows for high reliability, security, and efficiency, enabling the system to handle large amounts of data and user interactions.
- **System Integration:** PETURA's modules, such as incident reporting, adoption management, and notifications, are designed to integrate seamlessly. The use of APIs and modular design supports future expansion and integration with external services if needed.
- **Scalability and Performance:** With modular design principles and the use of Spring Boot, the system is capable of scaling horizontally by adding additional server instances to handle more users and data. This scalability supports future growth in shelter activities and user engagement.
- **Data Security:** The use of Spring Security ensures that the system enforces strong authentication and authorization practices, aligning with industry standards for data protection.

3.2 Economical Feasibility

The economic feasibility focuses on assessing whether the project's benefits justify the costs:

- **Development Cost:** PETURA is developed using open-source technologies (Spring Boot, Thymeleaf, SQL), which minimizes software licensing costs. Development and maintenance can be managed efficiently, especially with a small team, due to the readily available resources and community support for these technologies.
- **Operational Cost:** Hosting the application on a cloud-based platform or local server would be affordable, as cloud services like AWS and Azure offer scalable pricing models that adjust with usage. Additionally, PETURA's email notifications and map functionalities can be managed with minimal third-party service costs.
- **Benefit Analysis:** The cost-effectiveness of PETURA allows shelters to automate processes that would otherwise be time-consuming and labor-intensive, reducing overall operational costs. The return on investment is realized through improved efficiency, community support, and higher adoption rates.

3.3 Operational Feasibility

PETURA's operational feasibility considers how well the system will meet the day-to-day needs of shelters and users:

- **User-Friendly Interface:** PETURA is designed with usability in mind, providing clear navigation and role-specific dashboards. This ensures that administrators, staff, and general users can efficiently perform their respective tasks, such as updating animal profiles, managing adoptions, or making donations.
- **Support for Multi-Role Access:** PETURA incorporates role-based access control, enabling specific functions based on the user's role (admin, staff, or general user). This feature promotes operational efficiency by allowing users to access only relevant functions.
- **Resource Requirements:** The application's maintenance and operational needs are manageable for small to medium-sized shelters, as the system provides automated notifications, centralized data management, and efficient user workflows. Staff training on PETURA will be straightforward due to the intuitive interface and clear documentation.

3.4 Legal and Ethical Considerations

PETURA's development and operation incorporate essential legal and ethical considerations to protect users' and animals' rights:

- **Data Privacy and Security Compliance:** PETURA is designed to comply with data protection regulations by securing user data and sensitive animal welfare information. Personal information of adopters and donors is securely stored and processed following GDPR-like principles, ensuring privacy and transparency.
- **Animal Welfare and Ethical Standards:** PETURA aligns with ethical guidelines in animal welfare by providing accurate, transparent information about animals and shelter operations. The platform encourages responsible adoption practices and offers educational content on animal care.
- **Legal Accountability in Incident Reporting:** By incorporating a location-based incident reporting feature, PETURA provides a structured way for users to report animal-related incidents. All reports are subject to shelter review to ensure ethical handling of reported cases, aligning with legal requirements for animal welfare.

4. TOOLS AND TECHNOLOGIES USED

4.1 Tools:

4.1.1 Spring Tool Suite (STS):

Spring Tool Suite (STS) is an integrated development environment (IDE) specifically tailored for developing applications based on the Spring Framework. Built on top of the widely popular Eclipse IDE, STS provides a set of tools and features that streamline Spring development, making it easier for developers to create Spring-based applications. Although STS is designed primarily for Java applications, it includes specialized support for Spring Boot, Spring Cloud, and other Spring technologies.

Key Features:

1. **Spring-Specific Support:** STS offers built-in tools that simplify tasks like creating, testing, and deploying Spring-based applications. This includes features for Spring Boot, Spring Cloud, and Spring Integration.
2. **Advanced Project Management:** STS provides project templates and wizards to set up various Spring projects. It automatically configures Maven and Gradle build files, making it easier to start a Spring application.
3. **Integration with Spring Boot:** STS is fully integrated with Spring Boot, enabling seamless management of Spring Boot projects. It allows for live reload during development and offers embedded Tomcat support for quick application testing.
4. **Debugging and Testing:** STS provides enhanced debugging capabilities and supports various testing frameworks like JUnit, TestNG, and Spring Test. The IDE also includes a Spring-specific debugger for easy troubleshooting of Spring-based applications.
5. **Dependency Management:** STS simplifies managing dependencies for Spring projects with tools that automatically suggest dependencies based on the project's needs.

Extensions:

1. **Spring Tools 4 for Eclipse:** The main extension that brings Spring-specific functionality to Eclipse, STS includes Spring-related wizards, editors, and integration with cloud-based tools.
2. **Integration with Other Frameworks:** While STS is primarily for Spring development, it integrates well with other tools like Apache Maven and Gradle, and it can be extended for use with other frameworks like JavaFX, Hibernate, and more.

Use: STS is the ideal choice for developers working with the Spring ecosystem. Its deep integration with Spring technologies and its customization options make it the preferred IDE for building microservices, web applications, and enterprise-level systems. The tight integration with the Spring Framework allows for a more streamlined development process, from configuration to testing and deployment.

4.1.2 Postman

Postman is a popular API development and testing tool that simplifies the process of developing, testing, and managing APIs. It allows developers to send HTTP requests to web services, inspect responses, and automate testing workflows. It supports all HTTP methods, including GET, POST, PUT, DELETE, and more, making it ideal for working with RESTful APIs.

Use: In my project, Postman is used to test and debug the API endpoints, ensuring that features like user registration, donation processing, and animal adoption work correctly. It helps simulate requests and check responses, making it essential for API testing before integrating with the front-end.

4.1.3 Git

Git is a distributed version control system widely used for managing source code during software development. It allows developers to track changes to the codebase, collaborate effectively, and manage different versions of the code. Git supports branching and merging, which facilitates team collaboration and helps avoid conflicts in the codebase.

Use: Git is used in my project to manage the source code, enabling version control. With Git, I can track changes to the project, revert to previous versions if necessary.

4.1.4 Maven:

Maven is a build automation and dependency management tool for Java projects. It simplifies project builds by managing dependencies, compiling source code, running tests, and packaging the application for deployment.

Key Features:

1. **Dependency Management:** Automatically downloads and manages project dependencies from the central repository.
2. **Project Builds:** Provides a standard process for building and packaging Java projects.
3. **Plugins and Goals:** Supports a wide range of plugins for compiling code, running tests, generating reports, and more.
4. **Multi-Module Projects:** Handles projects with multiple modules and dependencies.

Use: Maven is used in PETURA to manage the dependencies like Spring Boot, JPA, and JavaMail. It automates the build process and ensures that all required libraries are available during development.

4.2 Technologies

4.2.1 Frameworks

4.2.1.1 Spring Boot

Spring Boot is a framework that simplifies the process of developing Java-based applications, particularly web applications, by providing a set of tools and configurations. It is built on top of the Spring Framework, designed to make it easier for developers to set up, configure, and deploy Spring applications with minimal setup. Spring Boot eliminates much of the boilerplate code and configuration required by Spring, making it particularly popular for microservices development.

Key Features:

1. **Auto Configuration:** Automatically configures Spring beans based on the project dependencies, which significantly reduces the need for manual configuration.
2. **Embedded Servers:** Spring Boot comes with embedded servers (like Tomcat, Jetty, and Undertow), so we don't need to deploy our app to an external server.
3. **Production Ready:** Spring Boot includes production-ready features like health checks, metrics, and monitoring, making it suitable for real-world, large-scale applications.
4. **Spring Boot CLI:** A command-line interface that allows developers to run Spring-based applications from the command line.
5. **Simplified Dependency Management:** Spring Boot manages dependencies and versions for us, ensuring compatibility with Spring projects.

Use: In my application PETURA, Spring Boot is the backbone for building APIs and managing business logic for features like user management, adoption processes, donation systems, and shelter management. The auto-configuration, embedded servers, and production-ready features make it easy to build, deploy, and maintain the backend of my application.

4.2.1.2 JPA (Java Persistence API)

JPA is a specification for managing relational data in Java applications. It provides a set of guidelines for object-relational mapping (ORM), allowing developers to map Java objects to database tables and vice versa. JPA is typically used with an ORM framework like Hibernate to perform database operations in an object-oriented manner.

Key Features:

1. **Object-Relational Mapping:** Automates the process of storing Java objects in a relational database and retrieving them.
2. **Entity Management:** JPA uses `@Entity` annotations to define persistent classes.

3. **Query Language:** Supports JPQL (Java Persistence Query Language), a SQL-like query language for querying entities.
4. **Transaction Management:** Integrated with Java's transaction management to ensure data consistency.

Use: In PETURA, JPA is used to manage entities like `Animal`, `User`, `Donation`, and `Adoption`. It simplifies CRUD operations and abstracts complex database interactions, allowing for efficient data management and queries.

4.2.1.3 Spring Security

Spring Security is a comprehensive security framework that provides authentication, authorization, and protection against common security threats. It integrates with Spring applications to enforce security rules for accessing resources, ensuring the safety of sensitive data.

Key Features:

1. **Authentication:** Verifies user identities, supporting various authentication methods (form-based, JWT, OAuth2).
2. **Authorization:** Manages user roles and permissions to control access to different parts of the application.
3. **Protection Against Attacks:** Guards against threats like CSRF (Cross-Site Request Forgery), XSS (Cross-Site Scripting), and session fixation.
4. **Customizable Security Rules:** Offers flexible ways to define security requirements for different URL patterns and resources.

Use: Spring Security is used to ensure that only authorized users (e.g., admins, staff) can access and modify data in your PETURA. It manages login, role-based access control, and protects the system from unauthorized access.

4.2.2 Database

4.2.2.1 MySQL

MySQL is an open-source relational database management system (RDBMS) used for storing and managing data. It utilizes Structured Query Language (SQL) for database access and manipulation. MySQL is widely used in web applications and supports large-scale, data-driven websites. It is known for its speed, reliability, and ease of use.

Key Features:

1. **Relational Database:** Organizes data into tables that relate to each other, making it easy to manage large datasets.

2. **ACID Compliant:** Ensures data integrity and supports transactions with properties like Atomicity, Consistency, Isolation, and Durability.
3. **Scalability:** Handles large amounts of data and supports horizontal scaling via clustering and replication.
4. **Cross-Platform:** Runs on various operating systems, including Linux, Windows, and macOS.
5. **Security:** Offers advanced security features like data encryption and access control.

Use: In the PETURA, MySQL is used to:

- **Store Animal and User Data:** MySQL maintains details like animal profiles, shelter information, and user accounts.
- **Manage Donations:** It tracks donor information, donation amounts, and recurring donation schedules.
- **Adoption Management:** It records adoption requests, animal availability, and transaction histories.
- **Reporting:** MySQL supports complex queries to generate reports on animal adoption, donation history, and shelter statistics.

MySQL is a key component for handling and retrieving structured data efficiently in the project.

4.2.3 Frontend

4.2.3.1 Thymeleaf

Thymeleaf is a modern server-side Java template engine used for rendering web pages in web applications. It is commonly used with Spring Boot to generate dynamic content in the form of HTML, XML, or other text-based formats. Thymeleaf is designed to be both server-side and capable of integrating seamlessly with modern web frameworks, providing the ability to work with dynamic data while keeping the view layer easy to maintain and flexible.

Key Features:

1. **Natural Templating:** Thymeleaf templates can be viewed as static HTML files in the browser, meaning that they can still be edited by designers without breaking the functionality. This is what's called "natural templating", which allows developers and designers to work independently without interfering with each other.
2. **Spring Integration:** Thymeleaf integrates seamlessly with the Spring Framework. It supports Spring MVC's model-view-controller architecture, making it a go-to choice for Spring Boot applications.
3. **Expressions and Loops:** Thymeleaf allows developers to embed expressions within templates, such as ``th:text``, ``th:each``, and ``th:if``, which allow dynamic rendering of data, looping through collections, and conditional rendering respectively.

4. **Layout and Fragmenting:** It supports modular templates by enabling the reuse of common fragments, like headers or footers, across different pages, which reduces duplication and increases maintainability.

Use: In PETURA, Thymeleaf is used for generating dynamic HTML content. For example:

- **User Management:** When a user logs in or registers, Thymeleaf dynamically renders the user dashboard page based on their role (admin, staff, or user).
- **Pet Adoption System:** The search and filter functionality on the pet adoption page can be dynamically rendered based on the criteria selected by the user.
- **Feedback Page:** After a user submits feedback, the confirmation page can be displayed dynamically using Thymeleaf's ability to render content based on server-side logic.

Thymeleaf enhances the user experience by allowing dynamic content to be displayed in real-time while ensuring the separation of concerns between the frontend (HTML) and the backend (Java), making the application more maintainable and scalable.

4.2.3.2 HTML (HyperText Markup Language)

HTML is the standard markup language used to create and design documents on the World Wide Web (WWW). It structures the content and elements of a web page, including text, links, images, videos, and interactive forms. HTML documents are interpreted by web browsers, which render the content to be displayed on users' screens.

Key Features:

1. **Structure:** HTML is used to structure content on the web using a system of elements (tags). Tags define the type of content, such as paragraphs, headings, links, and lists.
2. **Hyperlinks:** HTML allows the creation of hyperlinks (`<a>` tag), enabling navigation between different pages or resources on the internet.
3. **Multimedia Integration:** HTML supports embedding images (`<img>`), audio (`<audio>`), and video (`<video>`), enabling rich multimedia content.
4. **Forms:** It provides a range of form elements (`<form>`, `<input>`, `<textarea>`) to collect data from users, which is essential for user interaction in web applications.

Use: In PETURA, HTML is essential for building the frontend views. It is used to structure the layout of the webpages such as registration forms, pet-care tips, forums, feedback pages, etc. HTML, along with CSS (for styling) and JavaScript (for interactivity), forms the backbone of the user interface. It helps create a clean, readable, and functional web page, ensuring that users can easily interact with the system, whether they are looking to adopt pets, make donations, or learn about animal welfare.

By structuring the content with HTML, I ensure that the system is user-friendly and accessible across various devices and browsers.

4.2.3.3 CSS (Cascading Style Sheets)

CSS is a stylesheet language used to describe the presentation of a document written in HTML or XML. It defines how the elements of a webpage should be displayed, including aspects such as layout, colors, fonts, and spacing. CSS allows web developers to separate content from design, making it easier to maintain and manage the appearance of a website.

Key Features:

1. **Styling of Web Pages:** CSS provides control over the design of HTML elements, such as setting colors, fonts, margins, padding, alignment, etc. It enables developers to create visually appealing layouts and ensure a consistent look across multiple web pages.
2. **Responsive Design:** CSS enables responsive web design using media queries. This allows web pages to adapt their layout and content based on the screen size of the device being used, whether it's a desktop, tablet, or mobile phone.
3. **Flexbox and Grid Layouts:** CSS offers modern layout systems like Flexbox and Grid, which make it easier to create complex layouts that adjust to the size of the screen or container.
4. **Animations and Transitions:** CSS allows developers to create animations and transitions that enhance user interaction, such as hover effects or smooth page transitions when interacting with the website.

Use: In PETURA, CSS is used to style and enhance the user interface:

- **Dashboard and User Interface:** CSS is used to ensure the dashboard is visually appealing, making it easy for users to navigate between different modules, such as adoption, donation, and animal management.
- **Forms and Feedback:** The feedback form and other forms on my website can be styled using CSS to improve user experience, such as aligning input fields, styling buttons, and displaying error or success messages in a user-friendly way.
- **Responsive Design:** CSS media queries ensure that the site adapts to different screen sizes, making it accessible on mobile devices, tablets, and desktops, improving the usability of your platform.

CSS helps to define the overall user experience by PETURA visually appealing, responsive, and easy to use. It works seamlessly alongside HTML and JavaScript to create a dynamic, modern web application.

4.2.3.4 JavaScript

JavaScript is a high-level programming language used for building interactive and dynamic web pages. It enables client-side interactivity and allows the creation of rich, real-time applications. JavaScript is integral to web development, working alongside HTML and CSS to create responsive, dynamic user experiences.

Key Features:

1. **Interactivity:** Enables dynamic content updates without page reloads (e.g., real-time forms, notifications).
2. **Event-driven:** Triggers actions based on user input or other events.
3. **Asynchronous:** Supports operations that run in the background, enhancing performance (e.g., fetching data).
4. **Cross-platform:** Works across all modern browsers and devices.
5. **Server-side:** With Node.js, JavaScript can be used for backend development as well.

Use: In the PETURA, JavaScript enhances user experience by:

- **Form Validation:** Ensures user inputs are correct before submission.
- **Dynamic Content:** Updates animal search and donation results without refreshing the page.
- **User Interaction:** Powers interactive features such as modals and tooltips.
- **Real-time Updates:** Provides instant notifications for adoption or donation actions.

JavaScript is essential for creating an engaging, efficient, and responsive user interface in web applications.

4.2.4 Libraries Used

4.2.4.1 JavaMail

JavaMail API provides a platform-independent and protocol-independent framework for building mail and messaging applications. It enables Java applications to send and receive email messages over various protocols like SMTP, IMAP, and POP3.

Use: In PETURA, JavaMail is used for sending email notifications such as user registration confirmation, donation acknowledgments, adoption status updates, and incident alerts. It ensures that users and administrators are promptly notified of key events in the system, enhancing communication and user experience.

4.2.4.2 Jackson

Jackson is a high-performance JSON processor for Java, used for converting Java objects to JSON and vice versa. It is commonly used in web applications for handling data exchange between the frontend and backend.

Use: Jackson is used in PETURA to serialize and deserialize JSON data between the backend and the frontend. It ensures smooth communication and data transfer, particularly when handling API requests and responses for features like animal management, donations, and adoption processes.

4.2.4.3 Apache POI

Apache POI is a Java library used for reading and writing Microsoft Office documents, including Excel. It allows Java applications to create, modify, and display Excel files without needing Microsoft Office installed.

Use: Apache POI is utilized in PETURA to generate custom Excel reports. These reports provide insights into adoptions, donations, and other shelter activities. This feature allows shelters to analyze key metrics and trends, facilitating better decision-making and operational efficiency.

4.2.4.4 iText

iText is a library for creating and manipulating PDF documents in Java. It allows developers to generate PDFs from scratch, modify existing ones, and create complex reports.

Use: In PETURA, iText is used to generate PDF reports that shelters can use for printing or saving important records, such as adoption records, donation receipts, and incident reports. It enables shelters to create printable and shareable documents directly from the system.

4.2.5 External Tools

4.2.5.1 Leaflet

Leaflet is an open-source JavaScript library for building interactive maps. It is lightweight, simple to use, and works well across all modern desktop and mobile platforms. Leaflet supports features like markers, popups, and layers, making it ideal for displaying geospatial data.

Use: In PETURA, Leaflet is used for integrating interactive maps into the incident reporting module. Leaflet's simplicity and customization options make it a perfect tool for presenting location-based data effectively in the system.

Chapter-5: SYSTEM REQUIREMENTS

5.1 Hardware Requirements

To ensure smooth operation of PETURA, the following hardware requirements are recommended for both development and production environments:

- **Processor**: Dual-core or higher (Intel i5 or equivalent recommended)
- **RAM**: Minimum 8 GB (16 GB recommended for better performance during development and testing)
- **Hard Disk**: 100 GB or more of free disk space (for storing application, database, logs, and generated reports)
- **Network**: Stable internet connection for API calls, and external integrations
- **Graphics**: Basic integrated graphics for web-based frontend (higher specs for advanced graphical applications)

5.2 Software Requirements

The following software is required for running and developing PETURA:

- **Operating System**: Windows 10/11, macOS, or Linux
- **Java Development Kit**: JDK 11 or higher (required for compiling and running the Spring Boot backend)
- **Database**: MySQL for storing user data, animal information, donations, and other system-related data
- **Web Browser**: Latest versions of Chrome, Firefox, or Safari for accessing the web application
- **Web Server**: Apache Tomcat (embedded within Spring Boot for development)
- **Email Service**: JavaMail API for sending notifications via email

5.3 Development Tools

The following tools are recommended for the development of PETURA:

- **Integrated Development Environment (IDE)**:
 - Spring Tool Suite (STS) (preferred for Spring-based development)
 - IntelliJ IDEA or Eclipse (alternative Java IDEs)
- **Version Control**:
 - Git for managing the source code and collaborating with team members
 - GitHub/GitLab for hosting repositories
- **Build Tool**:
 - Maven for managing project dependencies, building the application, and automating tasks

- **API Testing:**
 - o Postman for testing RESTful APIs and ensuring correct functionality before integration with frontend

5.4 External Libraries

PETURA integrates the following external libraries for additional functionality:

- **Spring Boot**: Framework for developing the backend and managing server-side logic
- **Thymeleaf**: Templating engine used for building dynamic web pages on the frontend
- **JavaMail**: For sending email notifications such as registration, donation acknowledgment, and adoption status updates
- **Jackson**: For JSON serialization and deserialization between frontend and backend
- **Apache POI**: For generating Excel reports on shelter data, adoption, and donation activities
- **iText**: For creating PDF reports for adoption records, donations, and incident reports
- **Leaflet**: For integrating interactive maps within the application, primarily used in the incident reporting module

Chapter-6: SOFTWARE REQUIREMENT SPECIFICATION (SRS)

6.1 Functional Requirements

Functional requirements describe the system's behavior and the specific functionality it must support. The following are the primary functional requirements for PETURA:

- 1. User Registration and Authentication:**
 - Users should be able to register by providing their name, email, phone number, and password.
 - Users must be able to log in to the system using their email and password.
 - Admins should be able to view, edit, or delete user accounts.
- 2. User Dashboard:**
 - Each user (Admin, Staff, or Regular User) must have a role-based dashboard displaying relevant statistics.
 - Admins should have access to reports, user management, and data on donations, animals, and adoptions.
- 3. Animal Management:**
 - Admins should be able to add, update, delete, and view details about animals.
 - Users should be able to search and filter animals by type, breed, location, and adoption status.
- 4. Adoption Process:**
 - Users should be able to view available animals for adoption, submit adoption requests, and track the status of their requests.
 - Admins and staff should be able to review and update adoption request statuses.
- 5. Donation Management:**
 - Users should be able to make one-time or recurring donations.
 - Admins should be able to view and manage donation history and issue donation receipts.
- 6. Incident Reporting:**
 - Users should be able to report incidents such as animal neglect or abuse, including location details on a map.
 - Admins should be able to review incident reports and take appropriate action.
- 7. Notifications:**
 - The system must send email notifications for key actions, such as registration, donation receipts, adoption status updates, and incident alerts.
 - Notifications should be sent to admins and relevant stakeholders.
- 8. Report Generation:**
 - Admins should be able to generate and download custom reports in Excel and PDF formats related to adoption, donations, incidents, and animals.
- 9. Search and Filtering:**

- Users should be able to search for animals, incidents, and donation histories with filtering options for various parameters (e.g., animal type, donation amount, incident status).

6.2 Non-Functional Requirements

Non-functional requirements specify the qualities or attributes of the system, focusing on how well the system performs its functions. The following non-functional requirements apply to PETURA:

1. Performance:

- The system must be able to handle at least 1,000 concurrent users without significant degradation in performance.
- Response time for user requests should not exceed 3 seconds under normal load conditions.

2. Scalability:

- PETURA should be scalable to handle an increasing number of users, animals, donations, and incidents. The system should be able to expand horizontally if needed, by adding more servers to handle increased traffic.

3. Security:

- The system must use encryption (e.g., SSL/TLS) to secure sensitive data, including user credentials and financial transactions.
- User passwords must be stored securely using hash algorithms like bcrypt.
- Access to sensitive data (e.g., animal details, donor information) should be restricted based on user roles and permissions.
- Admins must have a secure login process with multi-factor authentication.

4. Usability:

- The user interface should be simple, intuitive, and user-friendly to ensure that all users, regardless of technical expertise, can navigate and use the system effectively.
- The system should provide helpful error messages and guide users through the registration, donation, and adoption processes.

5. Reliability:

- The system must be available 24/7, with minimal downtime for maintenance.
- Automated backups of the database should be performed daily to ensure data integrity.

6. Compatibility:

- PETURA must be compatible with all modern web browsers, including Chrome, Firefox, and Safari.
- The system should be responsive and accessible on desktop, tablet, and mobile devices.

7. Maintainability:

- The code should be modular and well-documented to ensure that it can be easily maintained and extended in the future.
- Any new features or bug fixes should be easy to implement without disrupting the existing functionality.

8. Compliance:

- The system should comply with relevant legal and regulatory standards for data protection.
- All email communications must comply with anti-spam regulations.

Chapter -7: SYSTEM ARCHITECTURE

7.1 Overview of the Architecture

The architecture of PETURA follows a modular, layered approach with distinct components, each responsible for specific tasks. This design ensures flexibility, scalability, and maintainability. The system is built using the Spring Boot framework for backend services and Thymeleaf for frontend rendering. The system is organized into several key modules, including User Management, Animal Management, Adoption, Donations, Incident Reporting, Notifications, and Reporting.

The architecture follows the Model-View-Controller (MVC) pattern, where:

- Model represents the data and business logic (e.g., animals, users, donations).
- View is the presentation layer handled by Thymeleaf templates.
- Controller handles user requests and acts as a bridge between the model and view.

The system also integrates external services such as email notifications (via JavaMail), map integration (via Leaflet), and report generation (via Apache POI and iText). All modules communicate through APIs or direct method calls, ensuring separation of concerns and easy maintainability.

7.2 High Level Diagram

The high-level architecture diagram provides a visual overview of the various components of PETURA and how they interact. It can include elements such as:

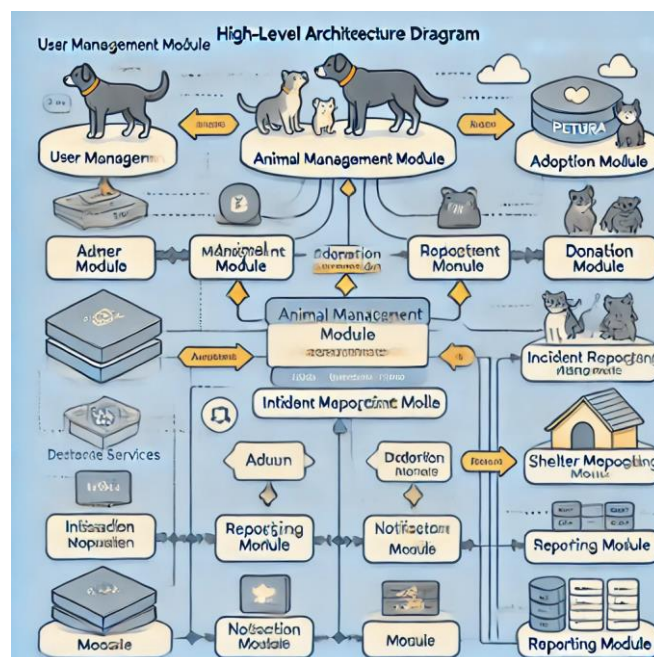


Fig: 7.1 High Level Architecture

1. **User Interface (Frontend):**

- The frontend, built with Thymeleaf, is responsible for rendering dynamic views, receiving user inputs, and displaying results.

2. **Web Layer (Controller):**

- The controllers handle HTTP requests and responses, processing user input and invoking backend services.

3. **Business Logic Layer:**

- The business logic layer processes application-specific tasks such as user registration, adoption workflows, donation processing, and incident reporting.

4. **Data Layer (Database):**

- The SQL database stores persistent data, including user details, animals, adoption requests, donations, and incident reports.

5. **External Services:**

- Email notifications (via JavaMail) for system events.
- Map integration (via Leaflet) for incident reporting.
- Report generation (via Apache POI and iText) for custom reports.

7.3 Component Design

The system is divided into several functional components that are loosely coupled and can be modified independently:

1. **User Management Module:**

- Responsible for user authentication, registration, and role-based access control (RBAC).

2. **Animal Management Module:**

- Handles CRUD operations for animals, including adding, updating, deleting, and viewing animal details.

3. **Adoption Module:**

- Manages the adoption process, including displaying available animals, submitting adoption requests, and tracking their status.

4. **Donation Module:**

- Handles one-time and recurring donations, donation history, and issuing receipts to donors.

5. Incident Reporting Module:

- Allows users to report incidents like animal abuse, and includes map integration to provide location data.

6. Shelter Management Module:

- Manages shelter-related tasks, including shelter creation, animal assignments to shelters, shelter status, and capacity management.

7. Notification Module:

- Sends email notifications for key events, such as user registration, donation acknowledgments, and adoption status updates.

8. Reporting Module:

- Supports custom report generation in Excel and PDF formats, including shelter-related reports for tracking animal intake, shelter conditions, and adoption metrics.

7.4 Data Flow

The data flow within the system follows a typical request-response cycle, with data being passed between various components. Here's an example of the data flow for a typical process:

1. User Registration:

- The user submits their registration details via the frontend (view).
- The controller receives the request, validates the input, and saves the user data in the database.
- After successful registration, the system sends an email notification to the user and admin via JavaMail.

2. Adoption Request:

- A user submits an adoption request via the adoption page.
- The backend controller receives the request, updates the adoption status in the database, and sends notifications to relevant staff members.

3. Donation Process:

- A user makes a donation through the donation page.
- The donation service records the donation in the database, updates the user's donation history, and triggers a donation receipt email via JavaMail.

4. **Incident Report:**

- A user reports an incident, including location data (using Leaflet).
- The incident service stores the report in the database and sends an email notification to relevant stakeholders for further action.

The data flow ensures that each component operates independently but communicates effectively through clearly defined interfaces and data models.

Chapter- 8: DETAILED DESIGN

8.1 Flowchart

8.1.1 User Authentication

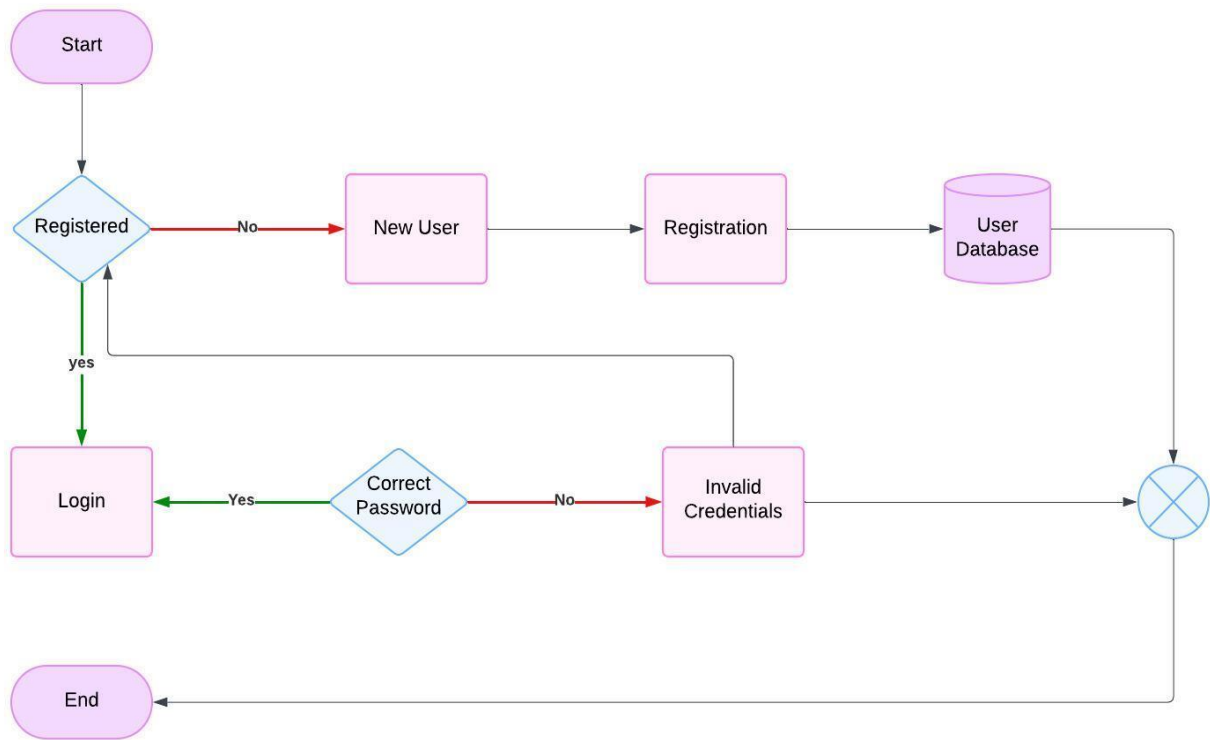


Fig: 8.1

8.1.2 Dashboard

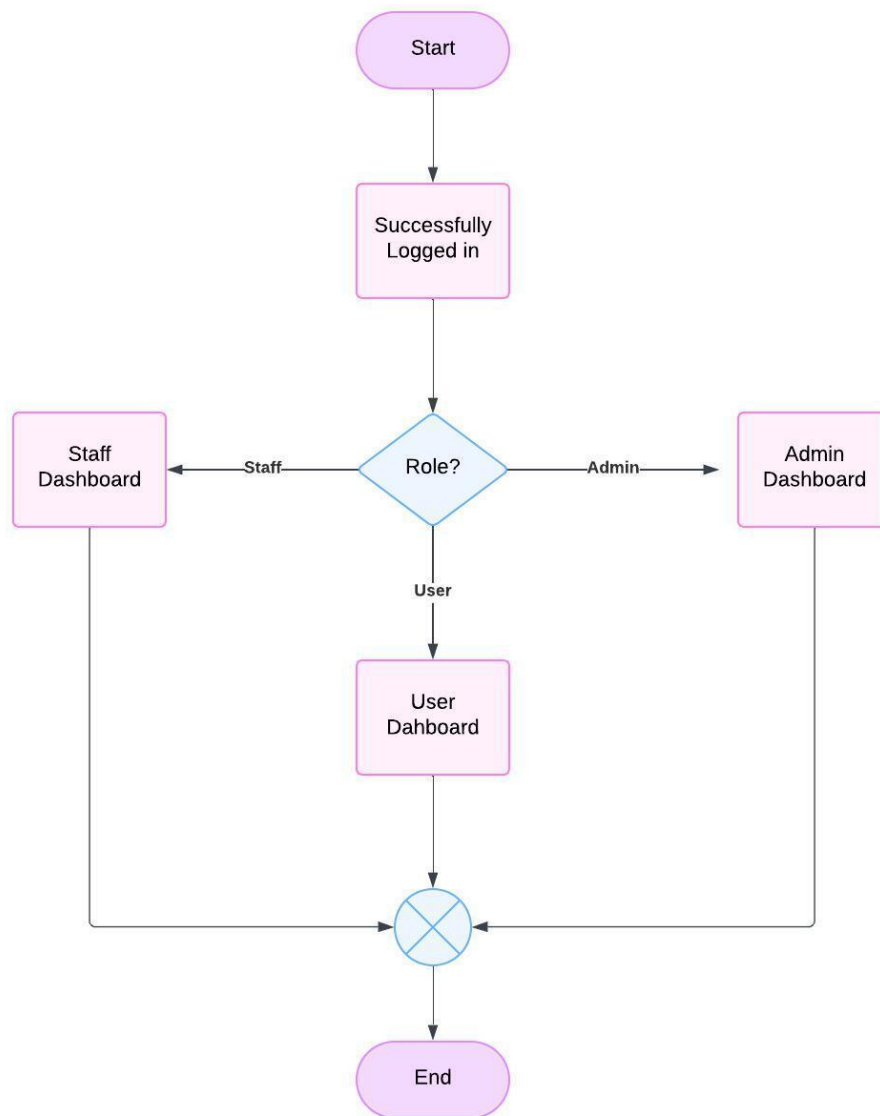


Fig: 8.2

8.1.3 Animal Module

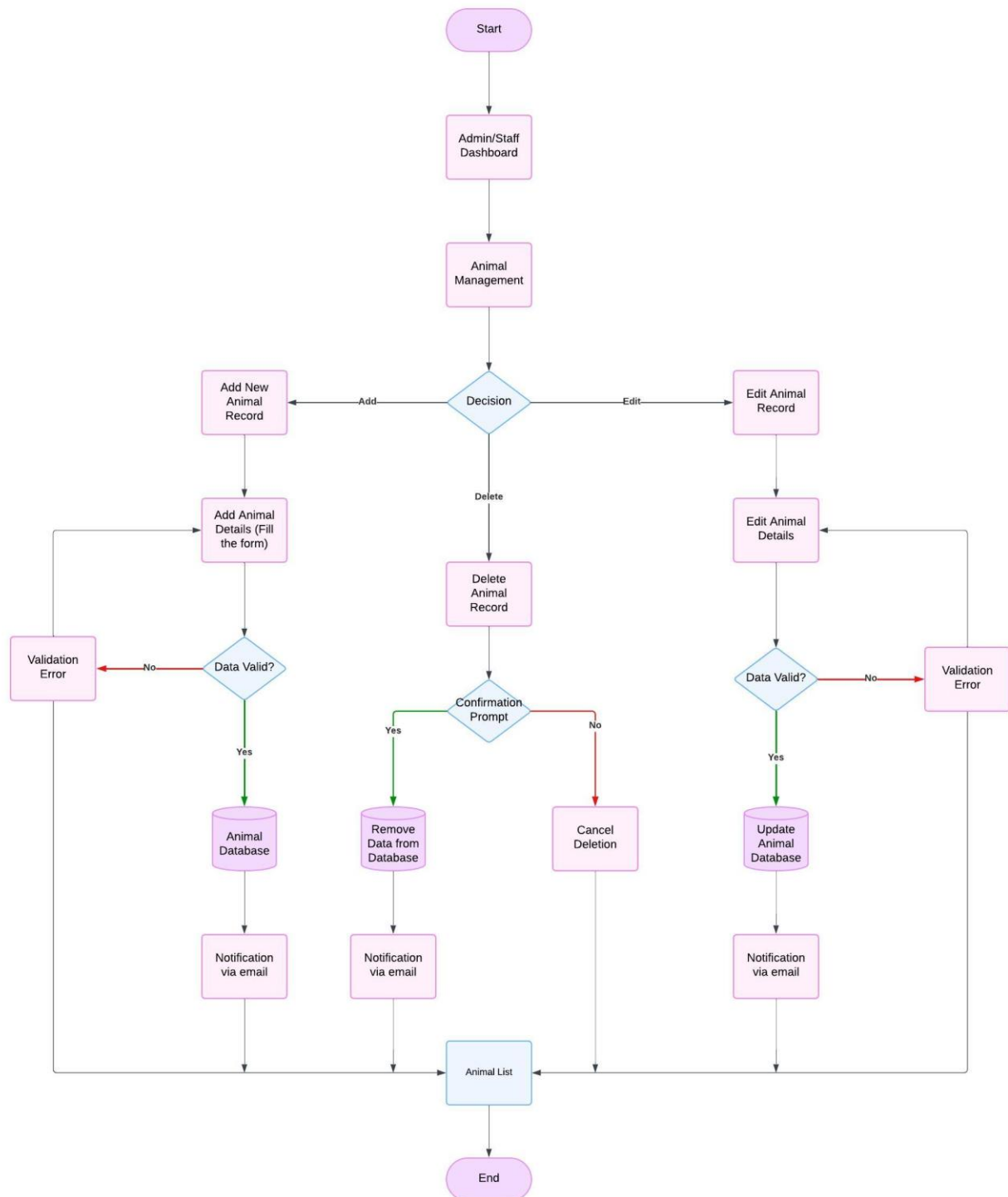


Fig: 8.3

8.1.4 Shelter Module

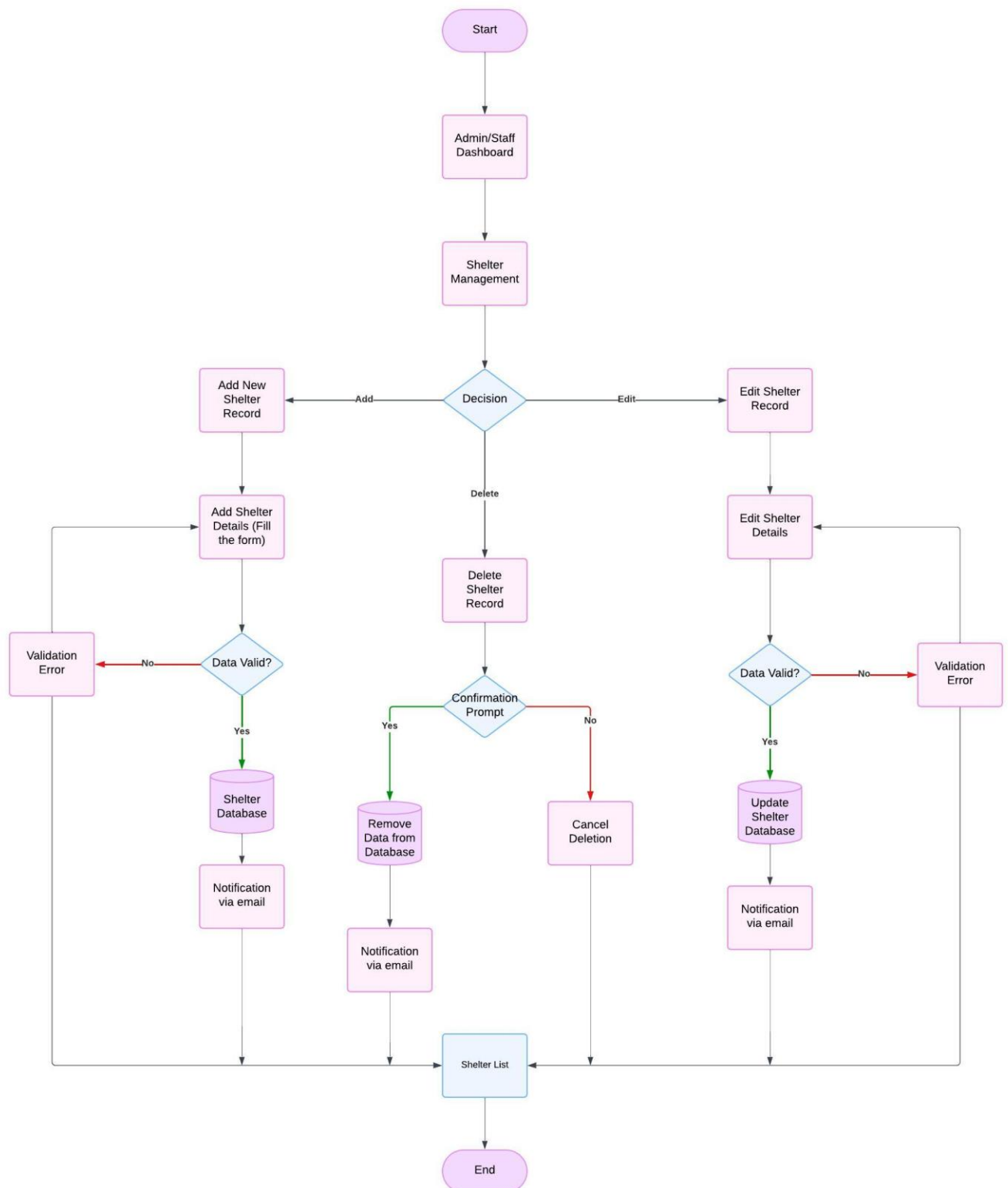


Fig: 8.4

8.1.5 Adoption Module

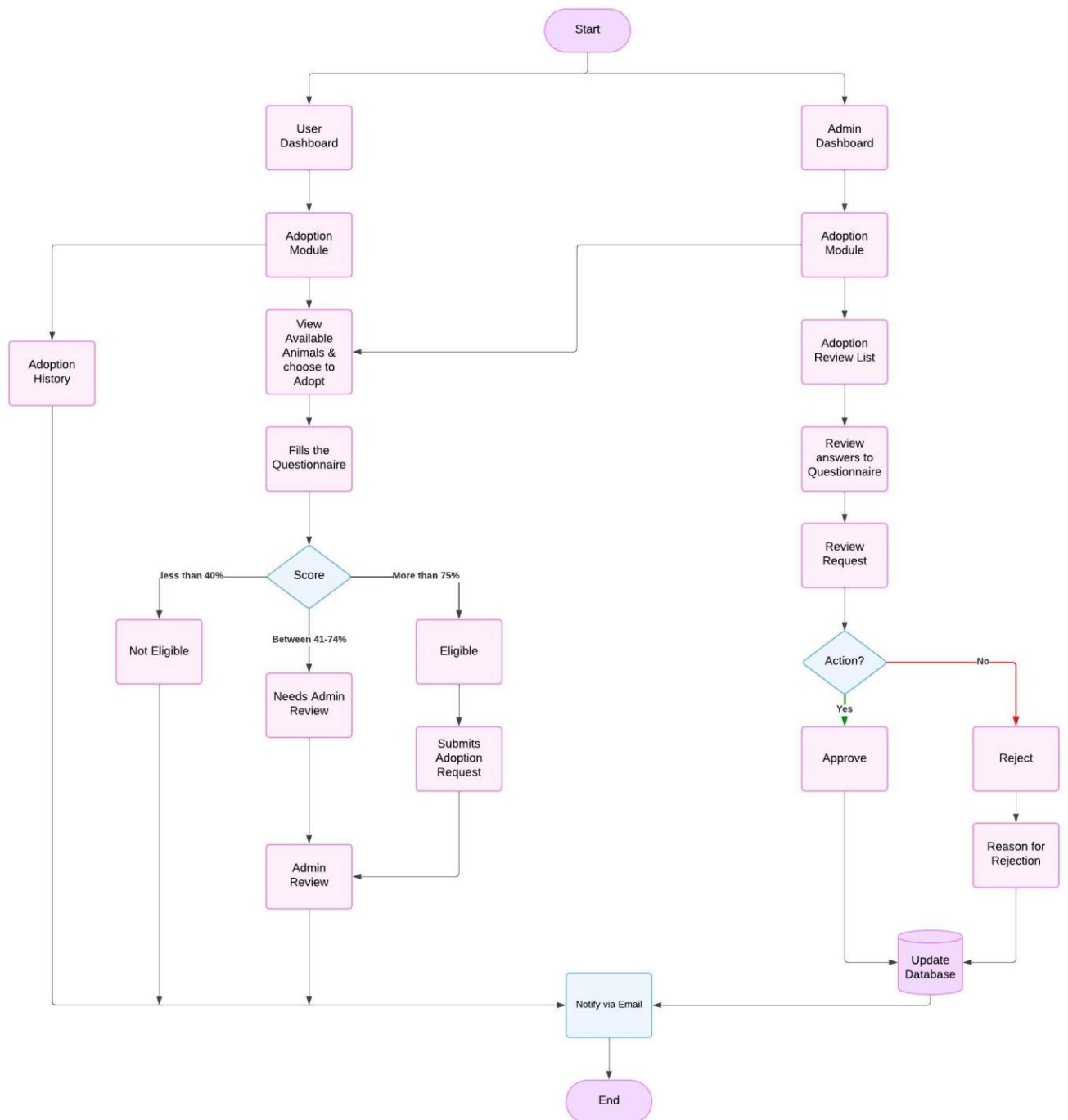


Fig: 8.5

8.1.6 Donation Module

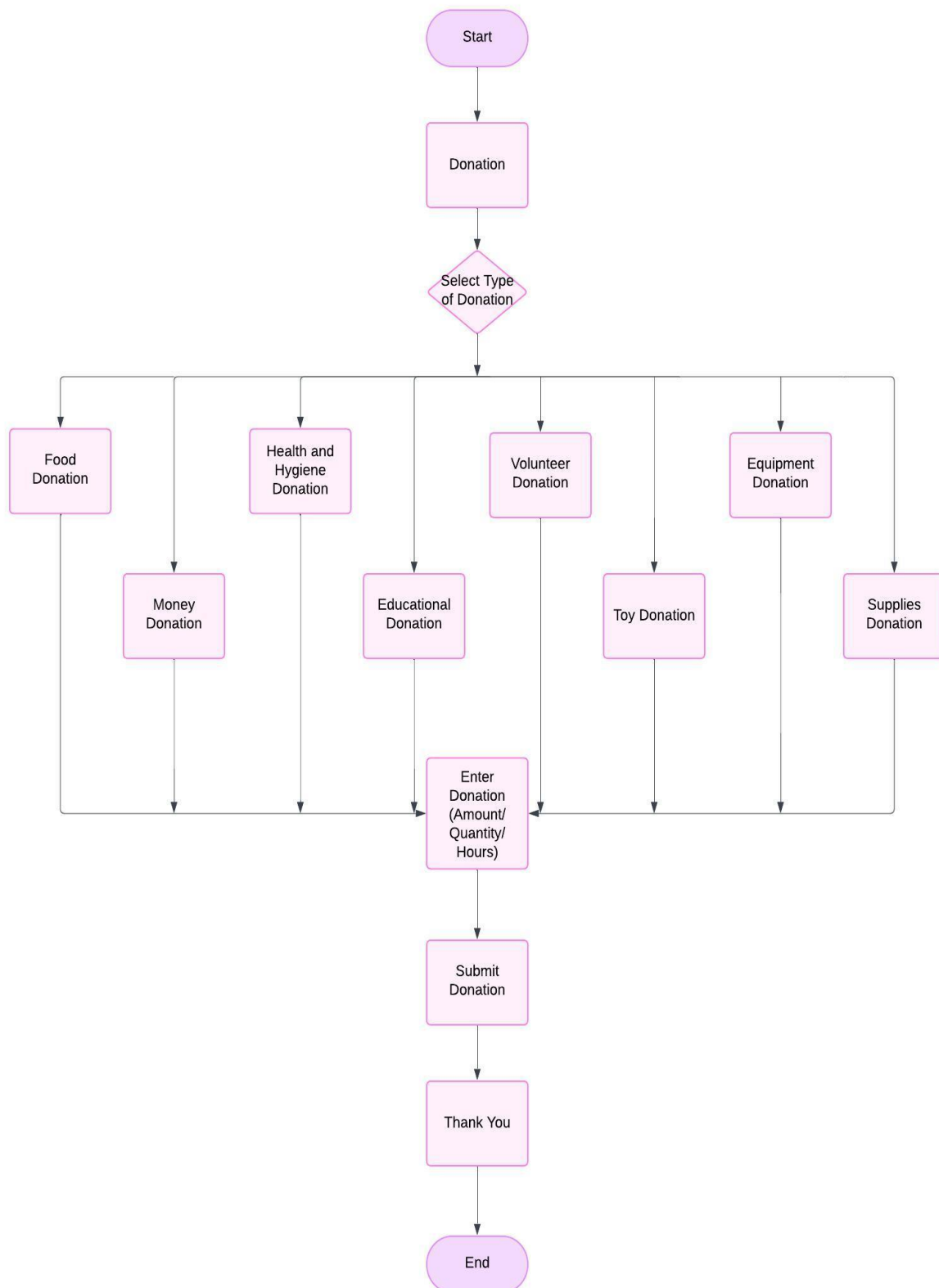


Fig: 8.6

8.1.7 Report Generation Module

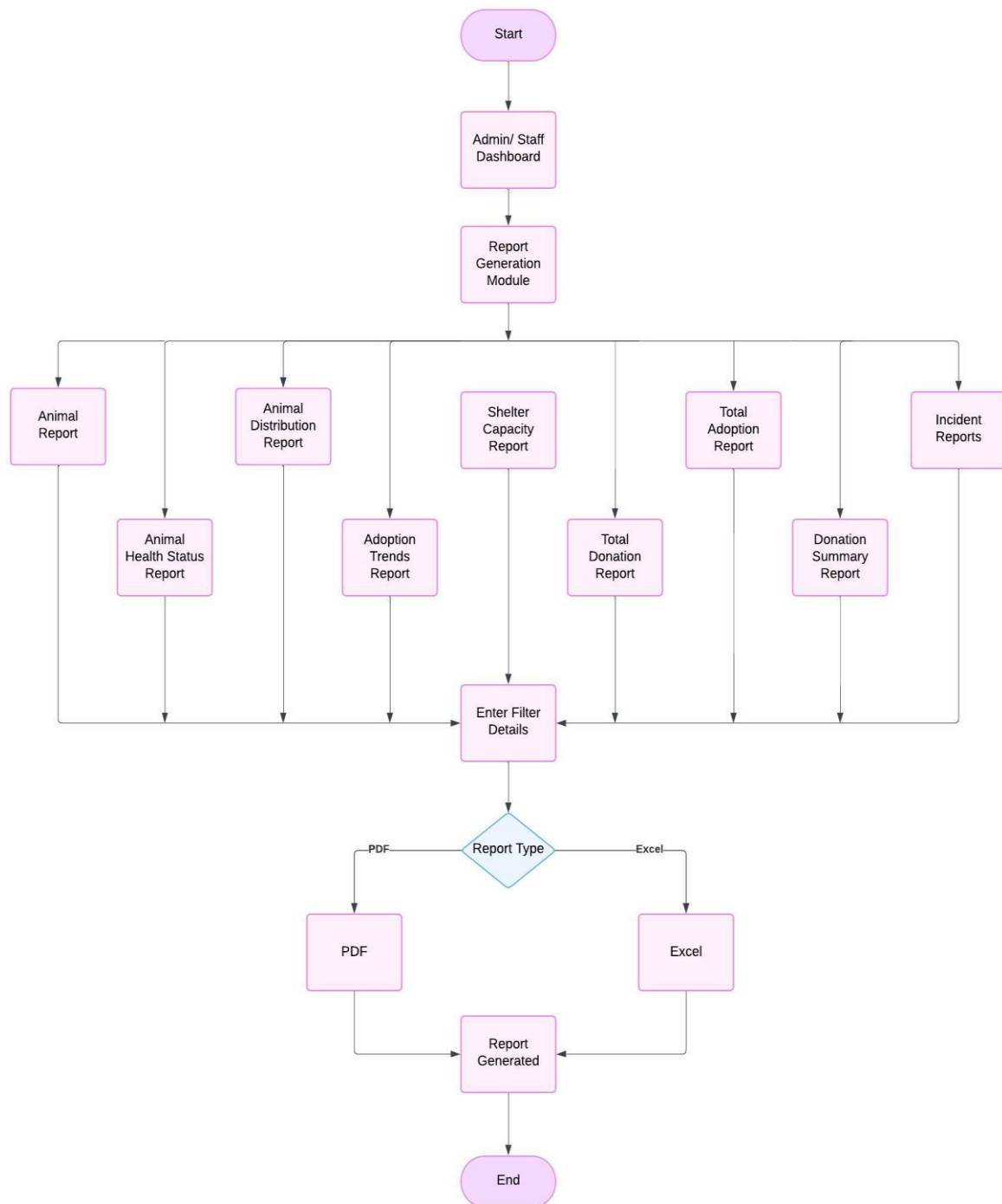


Fig: 8.7

8.1.8 Incident Report Module

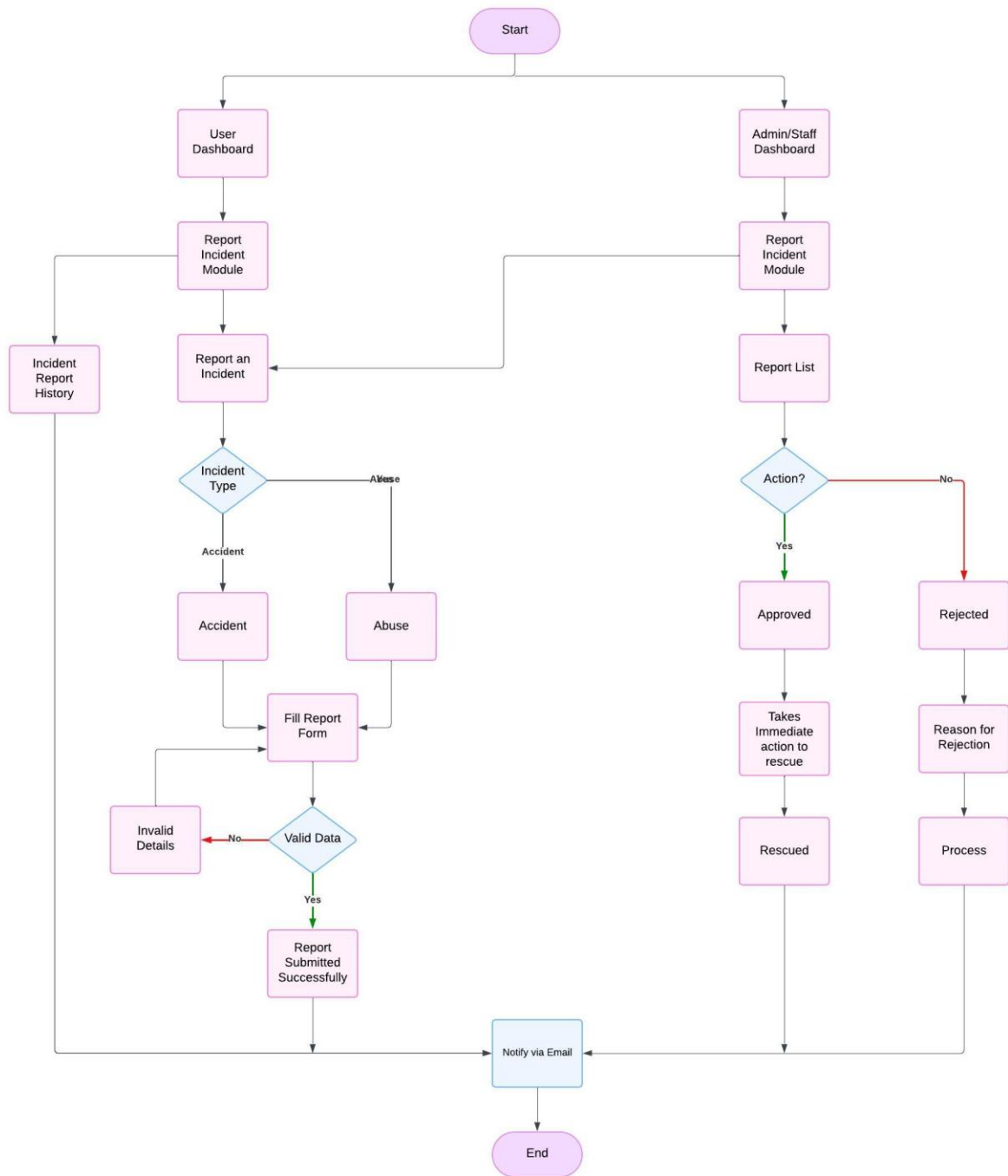


Fig: 8.8

8.2 Use Case Diagram

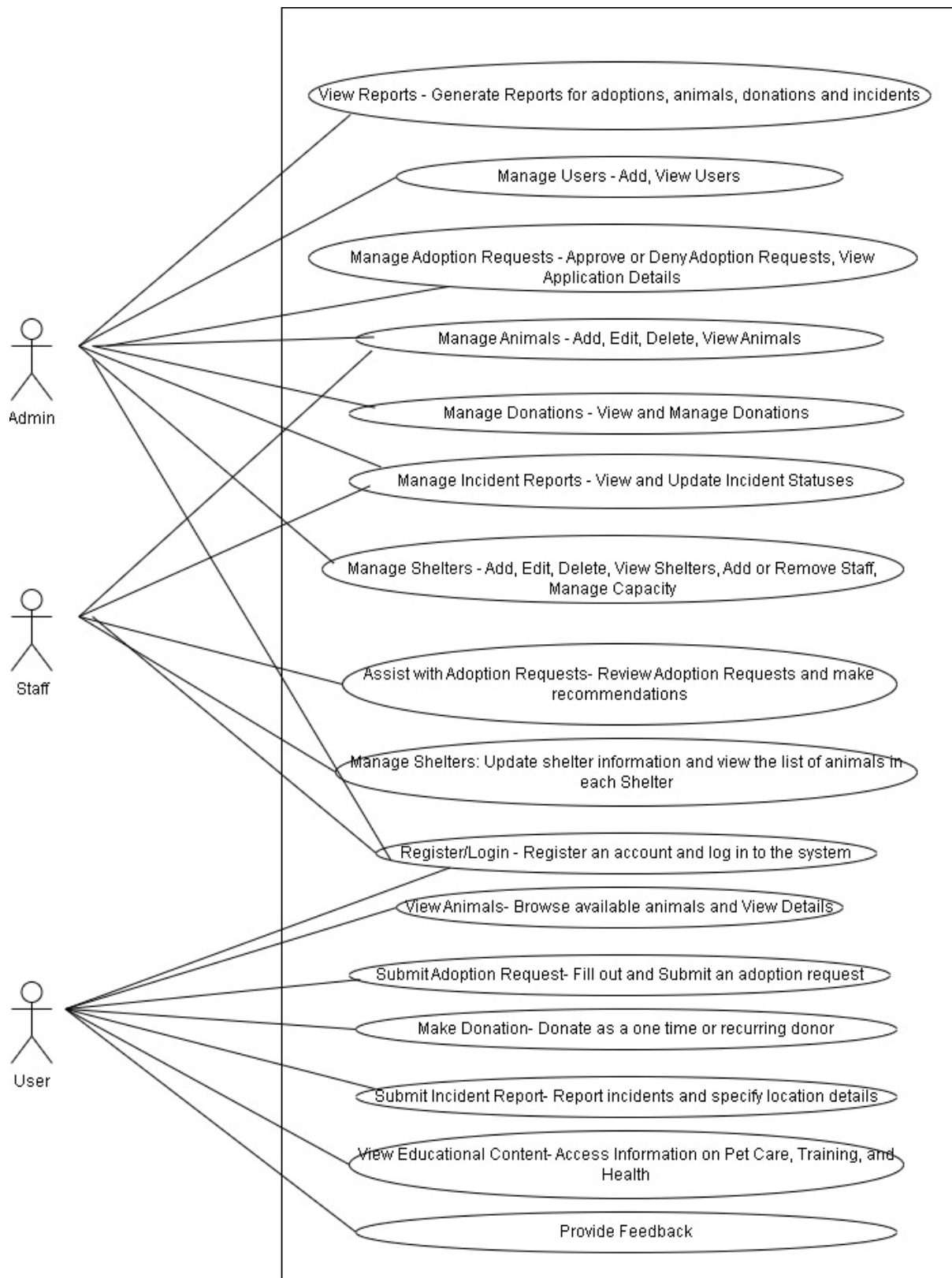


Fig: 8.9

8.3 Data Flow Diagram (DFD)

DFD Level-0

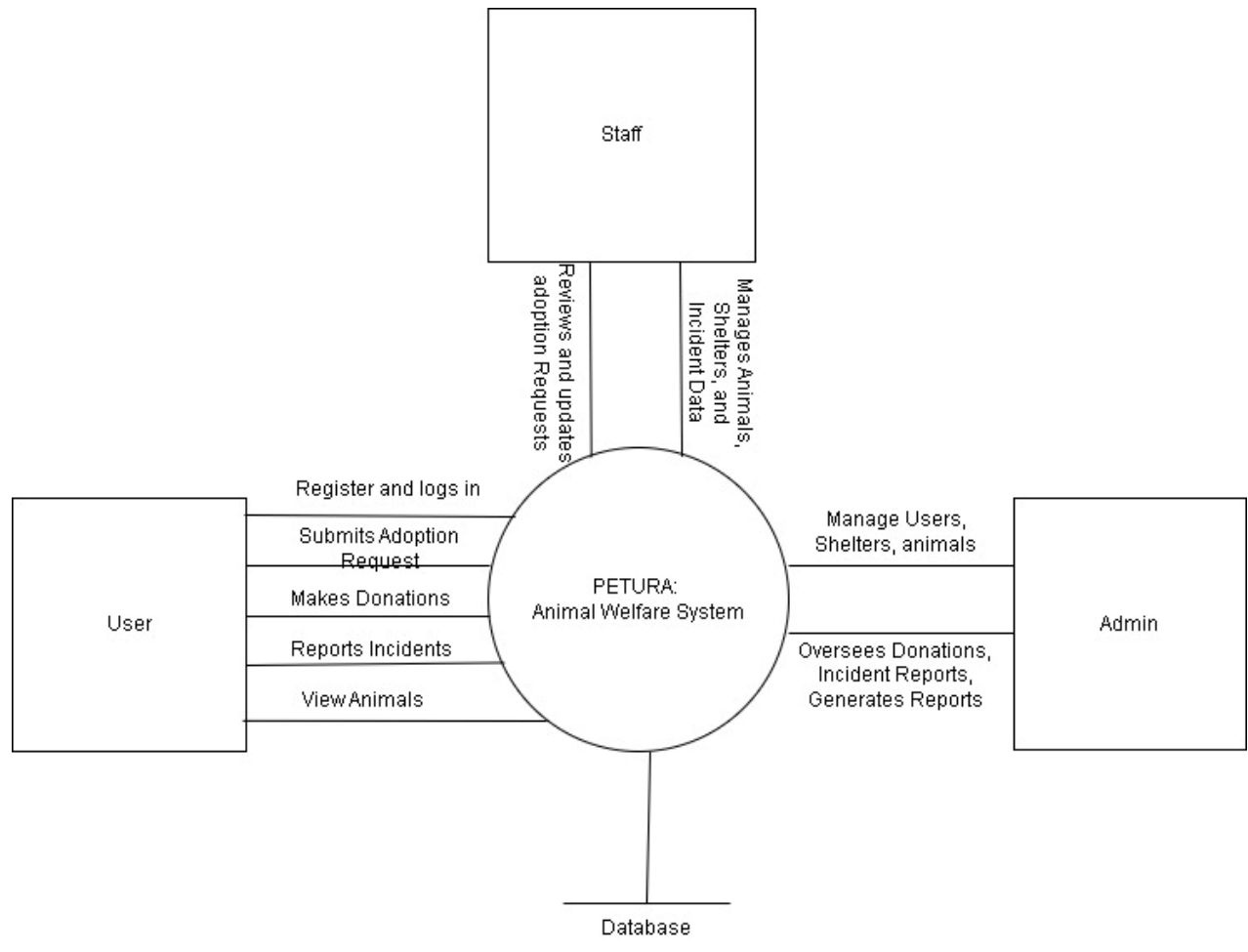


Fig: 8.10

DFD Level-1

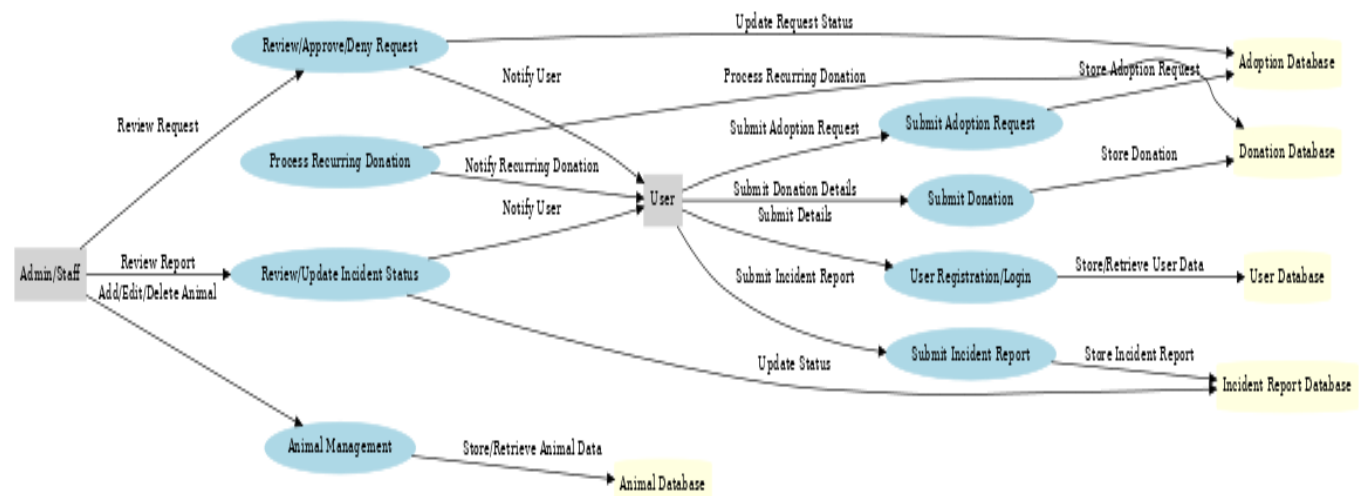


Fig: 8.11

DFD Level-2

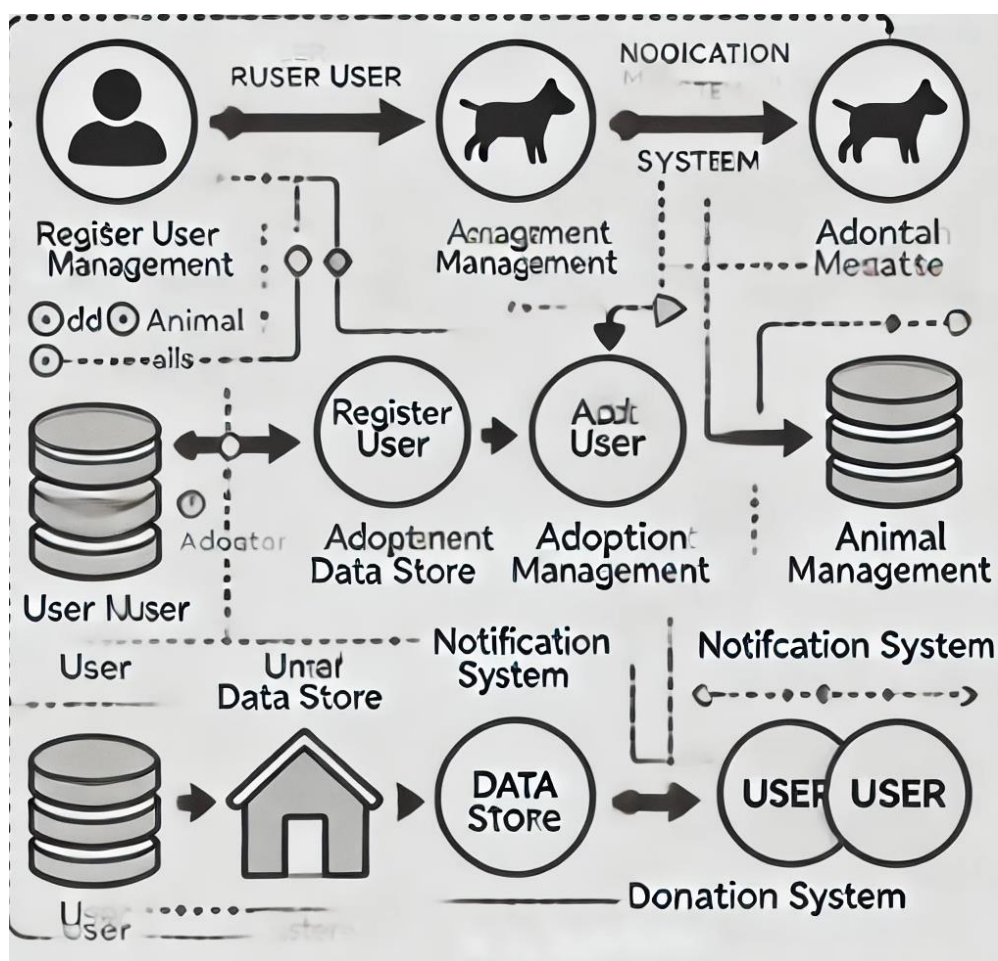


Fig: 8.12

8.4 Entity Relationship Diagram (ERD)

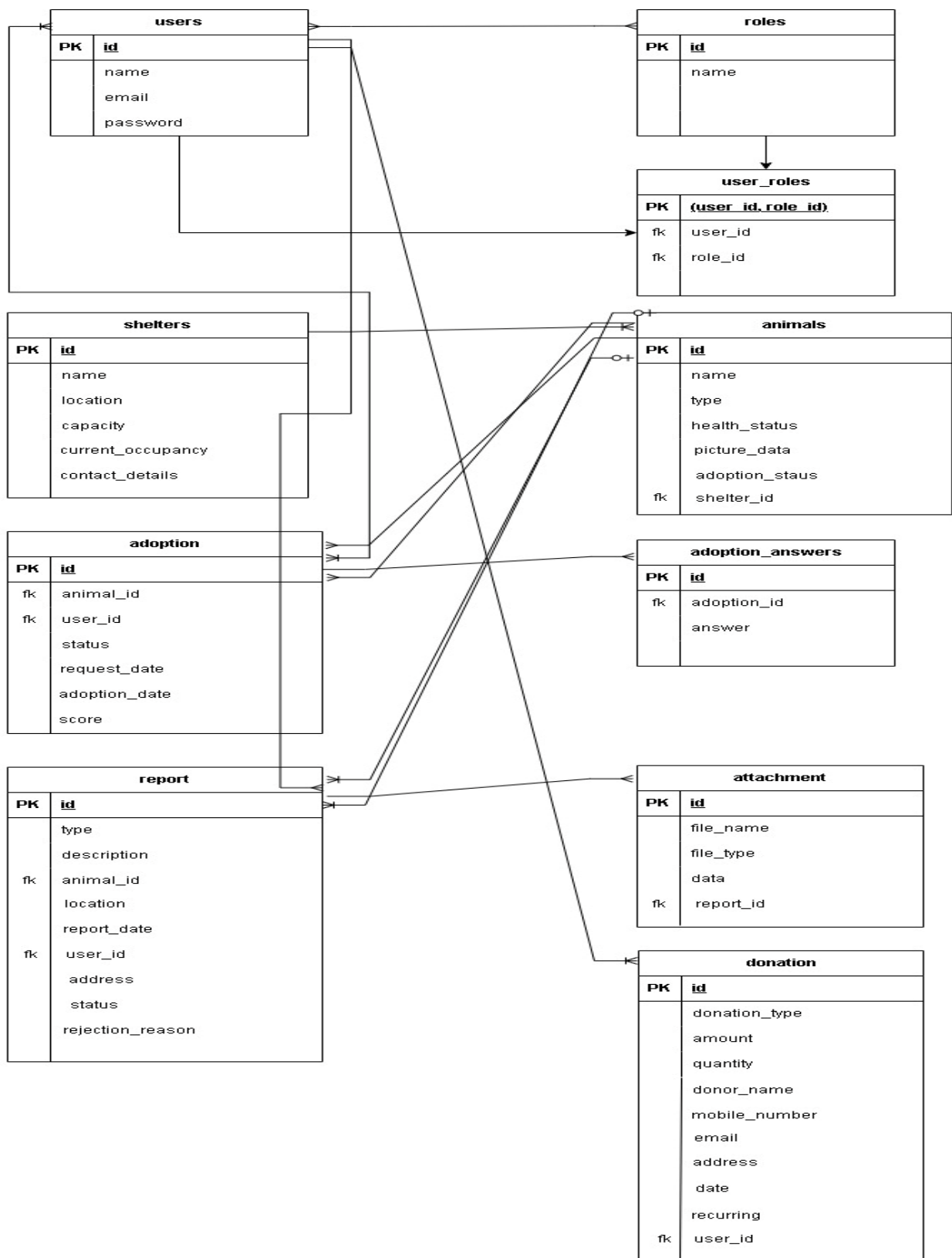


Fig: 8.13

8.5 Database Schema

```
CREATE database animalwelfaresyst
```

```
USE animalwelfaresyst
```

```
### 1. **User and Role Management**
```

```
) CREATE TABLE users (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    email VARCHAR(255) NOT NULL UNIQUE,  
    password VARCHAR(255) NOT NULL  
);  
  
) CREATE TABLE roles (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255) NOT NULL UNIQUE  
);  
  
) CREATE TABLE users_roles (  
    user_id BIGINT,  
    role_id BIGINT,  
    PRIMARY KEY (user_id, role_id),  
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
    FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE  
);
```

```
### 2. **Shelter Management**
```

```
⊖ CREATE TABLE shelters (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    location VARCHAR(255) NOT NULL,  
    capacity INT NOT NULL,  
    current_occupancy INT,  
    contact_details VARCHAR(255)  
);
```

```
### 3. **Animal Management**
```

```
⊖ CREATE TABLE animals (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    type VARCHAR(255) NOT NULL,  
    health_status VARCHAR(255) NOT NULL,  
    picture_data LONGBLOB,  
    shelter_id BIGINT,  
    adoption_status ENUM('AVAILABLE', 'ADOPTED', 'NOT_AVAILABLE') DEFAULT 'AVAILABLE',  
    FOREIGN KEY (shelter_id) REFERENCES shelters(id)  
);
```

4. **Adoption Management**

```
) CREATE TABLE adoption (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    animal_id BIGINT NOT NULL,  
    user_id BIGINT NOT NULL,  
    status ENUM('PENDING', 'APPROVED', 'REJECTED') NOT NULL,  
    request_date DATE NOT NULL,  
    adoption_date DATE,  
    score INT NOT NULL DEFAULT 0,  
    FOREIGN KEY (animal_id) REFERENCES animals(id),  
    FOREIGN KEY (user_id) REFERENCES users(id)  
);  
  
) CREATE TABLE adoption_answers (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    adoption_id BIGINT NOT NULL,  
    answer VARCHAR(255) NOT NULL,  
    FOREIGN KEY (adoption_id) REFERENCES adoption(id) ON DELETE CASCADE  
);
```

5. **Incident Reporting**

```
) CREATE TABLE report (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    type VARCHAR(255) NOT NULL, -- Enum values 'ABUSE' or 'ACCIDENT'  
    description TEXT NOT NULL,  
    animal_id BIGINT, -- Foreign key to animal table, nullable  
    location VARCHAR(255) NOT NULL,  
    report_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,  
    user_id BIGINT NOT NULL, -- Foreign key to user table  
    address VARCHAR(255),  
    status VARCHAR(50) NOT NULL DEFAULT 'pending',  
    rejection_reason VARCHAR(255),  
    FOREIGN KEY (animal_id) REFERENCES animals(id),  
    FOREIGN KEY (user_id) REFERENCES users(id)  
);  
  
) CREATE TABLE attachment (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    file_name VARCHAR(255) NOT NULL,  
    file_type VARCHAR(255) NOT NULL, -- e.g., image/jpeg, video/mp4  
    data LONGBLOB NOT NULL, -- Store the file data  
    report_id BIGINT NOT NULL, -- Foreign key to report table  
    FOREIGN KEY (report_id) REFERENCES report(id)  
);
```

6. **Donation Management**

```
CREATE TABLE donation (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  donation_type ENUM('FOOD', 'SUPPLIES', 'HEALTH_HYGIENE', 'TOYS', 'EQUIPMENT', 'MONEY', 'VOLUNTEER', 'EDUCATIONAL', 'OTHER') NOT NULL,  
  amount DECIMAL(10, 2),  
  quantity INT,  
  donor_name VARCHAR(255),  
  mobile_number VARCHAR(15),  
  email VARCHAR(255),  
  address VARCHAR(255),  
  date DATE,  
  recurring BOOLEAN NOT NULL DEFAULT FALSE,  
  user_id BIGINT,  
  FOREIGN KEY (user_id) REFERENCES users(id)  
);
```

Chapter-9: MODULES IN PETURA

9.1 User Module

Module Overview

The User Module is responsible for managing all user-related activities, including user registration, authentication, and profile management. It also handles role-based access control (RBAC), ensuring that users can access only the features they are authorized to use based on their role (Admin, Staff, User).

Key Features

- User registration and login
- Role-based access control (Admin, Staff, User)
- Profile management (viewing and updating user details)
- Admin notifications for new user registrations

Integration with Other Modules

- **Adoption Module**: Allows users to register and manage adoption requests after logging in.
- **Donation Module**: Users can donate to the shelter and view their donation history.

9.2 Animal Module

Module Overview

The Animal Module is responsible for managing the entire lifecycle of animals within the system, from registration to adoption. It allows admins and staff to add, update, delete, and view animals. It also supports animal search and filter functionality, helping users find animals based on specific criteria.

Key Features

- Animal registration (creating new animal profiles)
- Animal update and deletion
- Search and filter animals based on various attributes (e.g., species, breed, age)
- Admin notifications for animal creation, update, and deletion

Integration with Other Modules

- **Adoption Module**: Animals can be adopted after they are registered in this module.
- **Donation Module**: Donations can be directed toward specific animals or shelter needs.

9.3 Shelter Module

Module Overview

The Shelter Module helps in managing shelter operations, including shelter details, capacity, and animal allocation. Admins can manage shelters, add new shelters, update existing information, and track shelter capacity to ensure proper management of resources.

Key Features

- Shelter creation and management
- Animal allocation to shelters
- Shelter capacity tracking
- Admin notifications on shelter status changes

Integration with Other Modules

- **Animal Module**: Animals are assigned to specific shelters for proper care.
- **Donation Module**: Donations can be associated with specific shelters for resource management.

9.4 Adoption Module

Module Overview

The Adoption Module facilitates the entire adoption process, from registering adoption requests to reviewing and approving them. It includes features for users to apply for animal adoption and for admins to review, approve, or deny requests.

Key Features

- Adoption request submission by users
- Admin review and approval process
- Adoption eligibility questionnaire
- Email notifications for adoption status updates

Integration with Other Modules

- **Animal Module**: Animals that are available for adoption are listed in this module.
- **User Module**: Users can submit adoption requests after logging in.

9.5 Incident Report Module

Module Overview

This module allows users to report incidents involving animals or shelters, such as accidents, health issues, or abuse. It supports the tracking and resolution of incidents.

Key Features

- Incident report submission
- Incident tracking and resolution
- Map integration to mark incident locations
- Admin notifications upon report submission

Integration with Other Modules

- **Animal Module**: Reports related to specific animals can be filed here.
- **Shelter Module**: Incidents related to shelters can be tracked and resolved.

9.6 Donation Module

Module Overview

The Donation Module enables users to contribute funds to the shelter for animal care, shelter maintenance, or special projects. It supports both one-time and recurring donations and provides a seamless donation experience.

Key Features

- One-time and recurring donation options
- Donation tracking for users
- Donation acknowledgments (via email)
- Admin notifications for new donations

Integration with Other Modules

- **User Module**: Users can make donations through their accounts.
- **Shelter Module**: Donations can be directed to specific shelters or animal care needs.

9.7 Report Generation Module

Module Overview

The Report Generation Module allows the creation of custom reports based on system data, such as adoption statistics, donation history, and incident reports. Users can generate reports in Excel and PDF formats for analysis and presentation.

Key Features

- Custom report generation
- Export to Excel and PDF formats
- Report filters for customized data views

Integration with Other Modules

- **Animal Module**: Generate reports on animal statistics and adoptions.
- **Donation Module**: Create reports on donations made and total contributions.

9.8 Notification Module

Module Overview

The Notification Module handles the sending of notifications to users, admins, and staff. It supports email notifications for various system activities, such as registration, adoption status updates, donations, and incident reports.

Key Features

- Email notifications for user registration, donation acknowledgments, and adoption status
- Admin alerts for incidents, adoption requests, and donations
- Configurable notification templates

Integration with Other Modules

- **User Module**: Sends registration and login notifications.
- **Adoption Module**: Sends adoption status updates to users and admins.
- **Donation Module**: Sends thank-you emails for donations.

9.9 Search and Filter Module

Module Overview

The Search and Filter Module allows users to search for animals, donations, incidents, and shelters based on various attributes. It enhances user experience by providing easy navigation and finding relevant information within the system.

Key Features

- Search functionality for animals, incidents, shelters, and donations
- Advanced filtering options (e.g., by location, type, status)
- Fast and efficient search algorithms

Integration with Other Modules

- **Animal Module**: Provides filtering options for animal searches.
- **Shelter Module**: Allows users to search for shelters based on criteria such as capacity and location.

Chapter 10: SECURITY FEATURES

This chapter outlines the key security features implemented in the PETURA system, ensuring that the application is safe, user data is protected, and unauthorized access is prevented. The security architecture focuses on three primary areas: Authentication, Authorization, and Data Protection.

10.1 Authentication

Overview

Authentication is the process of verifying the identity of users before granting access to the system. PETURA uses Spring Security to manage the authentication process, ensuring that only registered users can access specific functionalities based on their roles.

Key Features

- **Login:** Users must authenticate by providing valid credentials (username/email and password). The system uses Spring Security's authentication mechanisms to validate credentials against the database.
- **Registration:** New users can register with their details (e.g., name, email, password) via the registration form. User passwords are securely hashed using BCrypt before being stored in the database.
- **Session Management:** After a successful login, a session is created for the user. The system uses session cookies to keep track of the user's logged-in state and automatically redirects them to the appropriate dashboard.

Authentication Flow

1. User enters login credentials (username and password).
2. The credentials are checked against the database.
3. If valid, a session is created, and the user is redirected to their appropriate dashboard.
4. If invalid, an error message is shown, and the user is prompted to try again.

10.2 Authorization

Overview

Authorization refers to the process of granting or denying access to specific resources or actions based on the user's role and permissions. PETURA uses Role-Based Access Control (RBAC) to ensure that each user can only access features that they are authorized to use based on their assigned role (Admin, Staff, User).

Key Features

- **Role-Based Access Control:** Users are assigned one of the following roles:
 - **Admin:** Has full access to all system functionalities, including managing animals, adoptions, donations, incidents, and users.
 - **Staff:** Has restricted access to certain modules, such as managing animals and incidents but not having full control over user management.
 - **User:** Has access to features related to adopting animals, making donations, and viewing their profile.
- **Role-Based UI Rendering:** The system dynamically adjusts the content visible on the user interface based on the user's role. For example, only admins can access the user management dashboard, while regular users will see only their adoption and donation history.
- **Access Control:** Specific actions within the system are protected by security annotations like `@PreAuthorize`` and `@Secured``, which check the user's role before allowing access to particular methods or views. This ensures that users cannot perform actions outside their permissions.

Authorization Flow

1. A user logs in and is assigned a role.
2. Based on the role, access to specific resources and actions is granted or denied.
3. Unauthorized users are redirected to an error page or denied access to restricted resources.

10.3 Data Protection

Overview

Data protection is crucial in ensuring the privacy and integrity of sensitive user information. PETURA follows best practices in securing user data, preventing unauthorized access, and safeguarding data both at rest and in transit.

Key Features

- **Password Security:** User passwords are securely hashed using BCrypt encryption before storing them in the database. This ensures that even if the database is compromised, passwords remain unreadable.
- **Secure Communication:** All communications between the user's browser and the server are encrypted using HTTPS (SSL/TLS). This ensures that sensitive data, such as login credentials and personal information, is transmitted securely over the network.
- **Data Storage:** Sensitive user data, such as email addresses, phone numbers, and donation details, are stored securely in the database. Access to this data is restricted to authorized users only. Sensitive information is encrypted in the database where necessary.
- **Session Management:** Sessions are handled securely using HTTPOnly cookies, which prevent JavaScript from accessing session information. Session timeouts and secure logout functionality are implemented to minimize the risk of session hijacking.
- **Access Logs and Auditing:** All actions performed by users, particularly those with admin access, are logged for auditing purposes. This allows for tracing any unauthorized access or malicious activities within the system.
- **Data Minimization:** The system collects only the data necessary for functionality, and sensitive data is not stored longer than required. For example, users can delete their accounts, which removes personal data from the system.

Data Protection Flow

1. User data is collected and securely stored in the database.
2. Sensitive data, such as passwords and personal information, is encrypted or hashed.
3. All data communications are encrypted using HTTPS to ensure secure transmission.
4. Users can manage their privacy settings and request data deletion, ensuring their rights to data protection.

Conclusion

By implementing robust authentication, authorization, and data protection measures, PETURA ensures that user data is safeguarded and that only authorized users have access to specific resources. These security features help protect the system from malicious attacks, unauthorized access, and data breaches, thereby enhancing the overall security of the application.

Chapter 11: USER ROLES AND PERMISSIONS

This chapter outlines the user roles and permissions in the PETURA system. It provides an overview of how Spring Security is configured to manage access to various functionalities based on the roles assigned to users. The system employs a Role-Based Access Control (RBAC) mechanism to ensure that users are granted access to specific features according to their roles, which include Admin, Staff, and User.

11.1 Overview of Spring Security Configuration

Spring Security is a powerful and customizable authentication and access-control framework used in the PETURA system to ensure secure access. The configuration defines user roles and restricts or grants permissions to certain endpoints or actions based on these roles.

Key Configurations

- **Role Hierarchy:** In PETURA, roles are hierarchical, with the `ROLE\_ADMIN` having the highest level of permissions, followed by `ROLE\_STAFF`, and then `ROLE\_USER`. This hierarchy allows for fine-grained access control.
- **HTTP Security Configuration:** Spring Security configures access restrictions through HTTP security settings in the backend. Specific URLs are protected with role-based access annotations, ensuring that only users with the correct role can access certain routes or resources.

```
package com.manmeet.animalsys.config;

import java.util.Set;

@Configuration
@EnableWebSecurity
public class SpringSecurity {

    @Autowired
    private UserDetailsServiceImpl userDetailsService;

    @Bean
    public static PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests((authorize) -> authorize
            .requestMatchers("/register/**", "/login", "/index", "/donations", "/donations/**", "/images/**",
                "/css/**", "/js/**", "/about", "/education", "/pet-care-tips", "/forum", "/forum/pet-care",
                "/forum/adoption", "/forum/volunteering", "/submit-feedback", "/feedback", "/thank-you", "/recent-donors")
            .permitAll()
            .requestMatchers("/dashboard", "/donations/recent-donors").authenticated() // All users can access the dashboard
            .requestMatchers("/users").hasRole("ADMIN") // Admin only access for users
            .anyRequest().authenticated() // All other requests require authentication
        ).formLogin(form -> form.loginPage("/login").loginProcessingUrl("/login").successHandler(customSuccessHandler())) // Custom
                                                                    // success
                                                                    // handler
            .permitAll().csrf().disable()
            .logout(logout -> logout.logoutRequestMatcher(new AntPathRequestMatcher("/logout")).permitAll());

        return http.build();
    }

    @Autowired
    public void configureGlobal(AuthenticationManagerBuilder auth) throws Exception {
        auth.userDetailsService(userDetailsService).passwordEncoder(passwordEncoder());
    }

    // Custom success handler for role-based redirection
    @Bean
    public AuthenticationSuccessHandler customSuccessHandler() {
        return (request, response, authentication) -> {
```

- **Custom Authentication and Authorization:** Custom filters can be used to implement specific logic for login, token-based authentication, or to restrict access further, depending on the application's needs.
- **Method Security:** Annotations like `@PreAuthorize` and `@Secured` are used at the method level to enforce access control. For example, `@Secured("ROLE_ADMIN")` ensures that a method can only be accessed by an admin.

11.2 Role-Based Dashboard

PETURA features a role-based dashboard that adjusts its content depending on the user's role. Upon logging in, the system checks the user's role and displays a dashboard that includes the relevant data and actions.

- Admin Dashboard:

The admin dashboard provides access to system-wide statistics, such as:

- Total animals, adoptions, and donations
- Active incidents and shelter data
- Reports and analytics
- User management functionality (e.g., create, update, delete users)

- Staff Dashboard:

The staff dashboard provides access to features like:

- Animal management (viewing, updating, and managing animal data)
- Incident report handling
- Viewing donation and adoption history

- User Dashboard:

The user dashboard includes features tailored to regular users, such as:

- Viewing available animals for adoption
- Making donations
- Accessing their donation and adoption history
- Profile management

Each dashboard is dynamically rendered based on the logged-in user's role, ensuring that users only see the options relevant to their access level.

11.3 Admin Role

The Admin role has the highest level of access in the PETURA system. Admins can manage all aspects of the system, from users to donations, animal management, and system reports.

Permissions

- User Management: Admins can create, update, or delete any user account.
- System Management: Full access to the administration of animals, shelters, adoptions, and donations.
- Reporting: Ability to generate reports on animals, donations, adoptions, incidents, etc.
- Configuration: Ability to configure system settings, such as email templates, notifications, and role assignments.

Admins can also access all sections of the application, including sensitive system areas like user authentication and permissions management.

11.4 Staff Role

The Staff role is designed for users who need access to certain parts of the system related to daily operations, such as animal management and incident reporting. Staff members do not have the full range of permissions that Admins have but can interact with essential features.

Permissions

- Animal Management: Staff can view, add, update, or delete animals in the system.
- Incident Reports: Staff can file, update, and review incident reports.
- Donations and Adoptions: Staff can view donation and adoption data but cannot delete or modify them.

Staff members do not have access to user management or system-level configurations, ensuring that sensitive operations are protected.

11.5 User Role

The User role represents regular users of the system who interact with features such as adopting animals, making donations, and viewing their personal information. Users have the least privileges and can only access features related to their own profiles.

Permissions

- Profile Management: Users can view and update their personal information.
- Adoption: Users can view available animals for adoption and submit adoption requests.

- Donations: Users can make one-time or recurring donations and view their donation history.

- Access to Reports: Users can access reports that pertain to their own donations and adoption history, but cannot generate system-wide reports.

Users cannot perform administrative or staff functions, such as modifying animal records, managing incidents, or generating complex reports.

11.6 Permission Table

Role	Permissions
Admin	Full access to all modules, including user management, animal management, reports, and system configuration.
Staff	Access to animal management, incident reporting, and viewing donation and adoption history.
User	Access to personal data, adoption requests, donation history, and making donations.

This table summarizes the key permissions associated with each role in the system. It helps to quickly visualize what each user can and cannot do.

Conclusion

The role-based security framework in PETURA ensures that each user has access only to the functionalities they are authorized for, protecting sensitive data and reducing the risk of unauthorized actions. By using Spring Security, the application ensures that roles and permissions are properly managed, allowing users to interact with the system based on their needs and responsibilities.

Chapter 12: SYSTEM WORKFLOW

This chapter outlines the workflows of the primary processes within the PETURA system, detailing the steps and interactions involved in key actions like adoption, donation, incident reporting, and animal management. Each process ensures a streamlined and efficient flow from the user's actions to the system's response, adhering to the business logic of PETURA.

12.1 Adoption Process

The adoption process in PETURA involves several steps to ensure that animals are adopted by suitable individuals. The system facilitates this process through a series of user and admin interactions, focusing on eligibility checks, adoption requests, and final approval.

Workflow

1. **View Available Animals:** Users browse available animals for adoption, filtering by type, age, size, and other characteristics.
2. **Submit Adoption Request:** Interested users fill out an adoption questionnaire, providing details about their suitability to adopt the animal.
3. **Eligibility Check:** The system validates the user's eligibility based on predefined criteria (e.g., previous adoption history, experience with pets).
4. **Review Request (Admin):** Admins receive the adoption request and review the user's eligibility.
5. **Approval/Rejection (Admin):** Admins approve or reject the adoption request based on the review. The user is notified of the decision.
6. **Adoption Finalization:** If approved, the adoption is finalized, and the animal's status is updated in the system.
7. **Notification:** Both the user and relevant staff members are notified of the adoption status via email.

12.2 Donation Process

The donation process allows users to contribute to the welfare of animals and shelters. Donations can be made on a one-time or recurring basis, and the system ensures smooth processing and acknowledgment of donations.

Workflow

1. **View Donation Options:** Users navigate to the donation page, choosing the amount and frequency (one-time or recurring).

2. **Enter Payment Details:** Users provide payment details.
3. **Process Donation:** The system processes the donation through an integrated payment gateway.
4. **Thank You Page:** After successful donation, a thank-you page is displayed to the user, acknowledging their contribution.
5. **Generate Receipt:** A donation receipt is generated and sent to the user via email for their records.
6. **Admin Notification:** Admins are notified of the new donation, allowing for internal tracking and reporting.
7. **Donation History:** The system updates the user's donation history, available for future reference.

12.3 Incident Reporting Process

The incident reporting process is a crucial part of the system, allowing users or staff to report incidents such as animal abuse, accidents, or any other emergencies. The report is tracked and escalated appropriately.

Workflow

1. **Submit Incident Report:** A user or staff member submits an incident report via the platform, providing details like the type of incident, description, location, and any other relevant information.
2. **Incident Review:** The report is reviewed by staff or admin, who assess the urgency and decide the next course of action.
3. **Assign to Relevant Authorities:** Depending on the severity of the incident, it is assigned to appropriate personnel or authorities for action (e.g., animal shelters, law enforcement).
4. **Track Incident Progress:** The incident is tracked within the system, updating status from "Reported" to "In Progress" to "Resolved."
5. **Incident Resolution:** Once the incident is resolved, a final update is made, and the report is closed.
6. **Admin and User Notification:** Both admins and users are notified of any updates or resolutions via email.

12.4 Animal Management Flow

The animal management flow encompasses the lifecycle of animal data within the system, from creation to updates and eventual removal or adoption.

Workflow

1. **Animal Data Entry:** Admins or staff members add new animal records, including essential information such as breed, age, medical history, and adoption status.
2. **Update Animal Records:** As animals are updated (e.g., change in status, health records), staff or admin update the records in the system.
3. **Animal Status Tracking:** Animals are tracked within the system based on their status (e.g., Available for Adoption, Adopted, In Shelter).
4. **Remove or Archive Animal Data:** Animals that are adopted or no longer part of the system are archived or deleted as per the workflow.
5. **Notification:** Admins are notified about significant changes, such as the addition or update of an animal's record.

Conclusion

These workflows represent the core processes of PETURA, ensuring a smooth and efficient system for adoption, donations, incident reporting, and animal management. By following these standardized workflows, PETURA can maintain an organized and responsive approach to animal welfare management, while also ensuring that users and staff can easily navigate and interact with the system.

Chapter 13: USER INTERFACE

This chapter outlines the design and functionality of the User Interface (UI) in PETURA. The interface is built to ensure a smooth and intuitive user experience, allowing users to easily navigate through the system's features and modules. Key elements such as the dashboard, pages, forms, and responsive design are covered in this section.

13.1 Dashboard

The Dashboard serves as the central hub for users of different roles, offering them a snapshot of relevant data and quick access to various features within the PETURA system. The dashboard is role-based, ensuring that users, staff, and admins have access to appropriate information and actions.

Features of the Dashboard

- **Role-Based Display:** Depending on the user's role (Admin, Staff, User), the dashboard dynamically adjusts to display relevant statistics, graphs, and actions.
- **Admin Dashboard:** Displays a comprehensive overview of system metrics, such as the total number of animals, pending adoption requests, recent donations, and incident reports.
- **Staff Dashboard:** Focuses on ongoing tasks such as managing animals, handling adoption requests, and processing incidents.
- **User Dashboard:** Provides the user with information on their adoption status, donation history, and personal updates.

Key Elements of the Dashboard

- **Navigation Sidebar:** Provides easy access to all modules, including Animal Management, Adoption, Donations, Reports, and more.
- **Quick Stats:** Displays key figures, such as total animals, adoptions, and donations, in real-time.
- **Actionable Buttons:** Depending on the role, users can take immediate actions (e.g., make a donation, apply for adoption, report an incident).

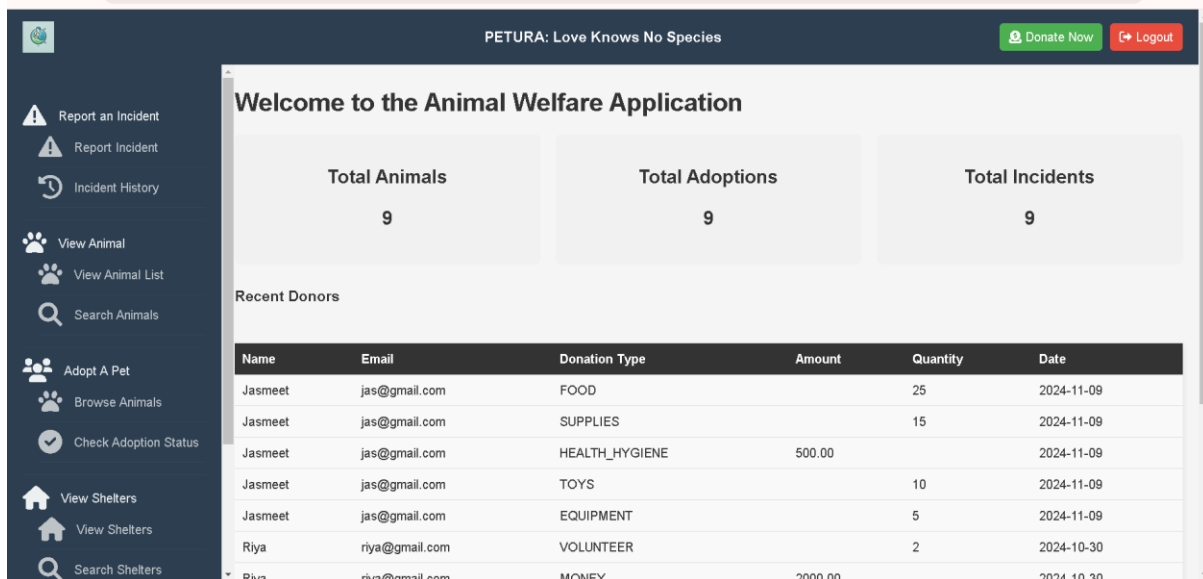


Fig:13.1: User Dashboard

13.2 Pages and Forms

Pages

PETURA includes several pages that serve specific functions such as viewing animals, submitting donation forms, applying for adoption, and viewing reports. These pages are designed to be user-friendly and consistent across the platform.

Forms

Forms are a critical part of the application, particularly in modules like Adoption and Donations. They are built with a user-centered design approach to simplify data input and improve user experience.

Key Form Features

- **Adoption Form:** Includes fields for personal information, preferences, and a series of eligibility questions to ensure the applicant meets adoption criteria.
- **Donation Form:** Allows users to select the amount, type of donation (one-time or recurring), and payment details.
- **Incident Report Form:** Lets users describe the incident, including location, type of incident, and any additional notes.

Form Validation and Error Handling

- **Real-Time Validation:** Provides feedback on incorrect inputs as users complete forms (e.g., required fields, valid email format, etc.).

- **Error Messaging:** Clear error messages are displayed when there are issues with form submission to guide the user in correcting their input.

Fig: 13.2: Incident Report Form

13.3 Responsive Design Approach

The Responsive Design approach ensures that PETURA provides an optimal user experience across all devices, including desktops, tablets, and smartphones. The interface adapts seamlessly to different screen sizes, providing a consistent and functional experience.

Key Principles of Responsive Design

- **Fluid Layouts:** The design adjusts fluidly based on screen size, ensuring that content is displayed proportionally on all devices.
- **Flexible Images and Media:** Images and media elements scale according to the size of the screen, ensuring that they are appropriately sized for each device.
- **Mobile-First Design:** The design prioritizes mobile users by optimizing elements for smaller screens and gradually enhancing for larger screens.
- **Media Queries:** CSS media queries are used to adjust the layout, font sizes, and visibility of elements based on device characteristics such as width, height, or orientation.

Responsive Behavior Examples

- **Navigation:** On small devices, the sidebar collapses into a hamburger menu, allowing users to access the navigation items.
- **Forms and Buttons:** Buttons and form fields resize to fit the screen width, making them easier to interact with on mobile devices.

- **Grid System:** The content on the dashboard and other pages is organized in a grid that adjusts dynamically to different screen sizes.

Conclusion

PETURA's User Interface (UI) is designed to be intuitive, responsive, and functional. It provides an optimized user experience for all roles and devices, ensuring that each user can easily interact with the system, whether they are adopting an animal, donating, or reporting an incident. The focus on responsiveness ensures that PETURA can be accessed efficiently on any device, providing flexibility and usability for all types of users.

Chapter 14: REPORTS AND ANALYTICS

This chapter discusses the reporting and analytics capabilities of the PETURA system. These features are essential for tracking key metrics, generating reports, and analyzing data to gain actionable insights. Reports help stakeholders make informed decisions about animal welfare, donations, adoptions, and more. The system provides robust custom report generation, data analysis tools, and visual insights.

14.1 Custom Report Generation (Excel & PDF)

The Custom Report Generation functionality in PETURA allows users to generate detailed and customizable reports based on the data available in the system. These reports can be exported in two formats: Excel and PDF, ensuring that users have flexibility in how they store, analyze, and share the information.

Key Features:

1. Excel Report Generation:

- **Data Selection and Filtering:** Before generating an Excel report, users can filter the data according to specific criteria. For example, users may choose a particular time range, animal type, donation category, or adoption status. This feature ensures that the report contains only the most relevant data.
- **Customizable Layout:** Reports can be designed with a flexible layout that includes various data fields such as donor names, donation amounts, animal details, or adoption statuses. The data can be displayed across multiple sheets or tabs for better organization.
- **Multiple Data Categories:** The Excel reports can include multiple categories of data, such as:
 - **Animal Reports:** Including details such as animal type, shelter, health status, and adoption status.
 - **Donation Reports:** Tracking donation amounts, types (one-time or recurring), and donor information.
 - **Adoption Reports:** Information on adoption requests, their approval statuses, and the involved animals.
 - **Graphical Representations:** Users can include simple graphs or charts in the report to visually represent data, such as donation trends or animal adoption numbers. These visual elements are helpful for stakeholders who want to analyze the data at a glance.
- **Batch Processing:** For large amounts of data, Excel reports can be generated in batches to prevent delays or timeouts, especially when dealing with a large dataset like donation history or animal records.

	A	B	C	D	E	F	G
1	Donor Name	Amount	Date	Donation Type	Quantity	Recurring	
2	Riya	0	Wed Oct 3	VOLUNTEER	2	Yes	
3	Riya	2000	Wed Oct 3	MONEY	0	No	
4	Riya	1000	Wed Oct 3	HEALTH_H	0	No	
5	Riya	0	Wed Oct 3	TOYS	10	No	
6	Jasmeet	0	Sat Nov 09	FOOD	25	No	
7	Jasmeet	0	Sat Nov 09	SUPPLIES	15	No	
8	Jasmeet	500	Sat Nov 09	HEALTH_H	0	Yes	
9	Jasmeet	0	Sat Nov 09	TOYS	10	No	
10	Jasmeet	0	Sat Nov 09	EQUIPMENT	5	No	
11	Total Donations	9					
12							
13							
14							

Fig: 14.1: Excel Report

Exporting Reports: After creating the report, users can easily download it in Excel format. This makes it easy for them to further manipulate the data, perform calculations, or share it with other team members.

2. PDF Report Generation:

- **Professional Layouts:** PDF reports are designed to be formal, clean, and ready for distribution. They can include headers, footers, logos, and custom formatting to ensure that the reports are visually appealing and meet organizational standards.

- **Fixed Layout:** Unlike Excel reports, PDF reports have a fixed layout that ensures the data remains in place, even when viewed or printed. This is ideal for formal reporting or documentation purposes.

- **Customizable Sections:** PDF reports can be customized to include sections that summarize key insights, such as total donations, adoption rates, or animal welfare statistics. This makes it easy for readers to digest the most important information.

- **Data Formatting:** When generating a PDF report, the data is formatted in a clear, readable way. This may include sorting, highlighting key numbers, and ensuring consistent font sizes for readability.

Donation Summary

Donor Name	Amount	Date	Donation Type	Quantity	Recurring
Riya	0	Wed Oct 30 00:00:00 IST 2024	VOLUNTEER	2	Yes
Riya	2000.00	Wed Oct 30 00:00:00 IST 2024	MONEY	0	No
Riya	1000.00	Wed Oct 30 00:00:00 IST 2024	HEALTH_HYGIENE	0	No
Riya	0	Wed Oct 30 00:00:00 IST 2024	TOYS	10	No
Jasmeet	0	Sat Nov 09 00:00:00 IST 2024	FOOD	25	No
Jasmeet	0	Sat Nov 09 00:00:00 IST 2024	SUPPLIES	15	No
Jasmeet	500.00	Sat Nov 09 00:00:00 IST 2024	HEALTH_HYGIENE	0	Yes
Jasmeet	0	Sat Nov 09 00:00:00 IST 2024	TOYS	10	No
Jasmeet	0	Sat Nov 09 00:00:00 IST 2024	EQUIPMENT	5	No
Total Donations	9				

Fig: PDF Report

Exporting Reports: Similar to Excel reports, PDF reports can be downloaded and shared. The PDF format ensures that the report's layout remains intact, and it is suitable for stakeholders who require a non-editable, polished document.

3. Customization and Filters:

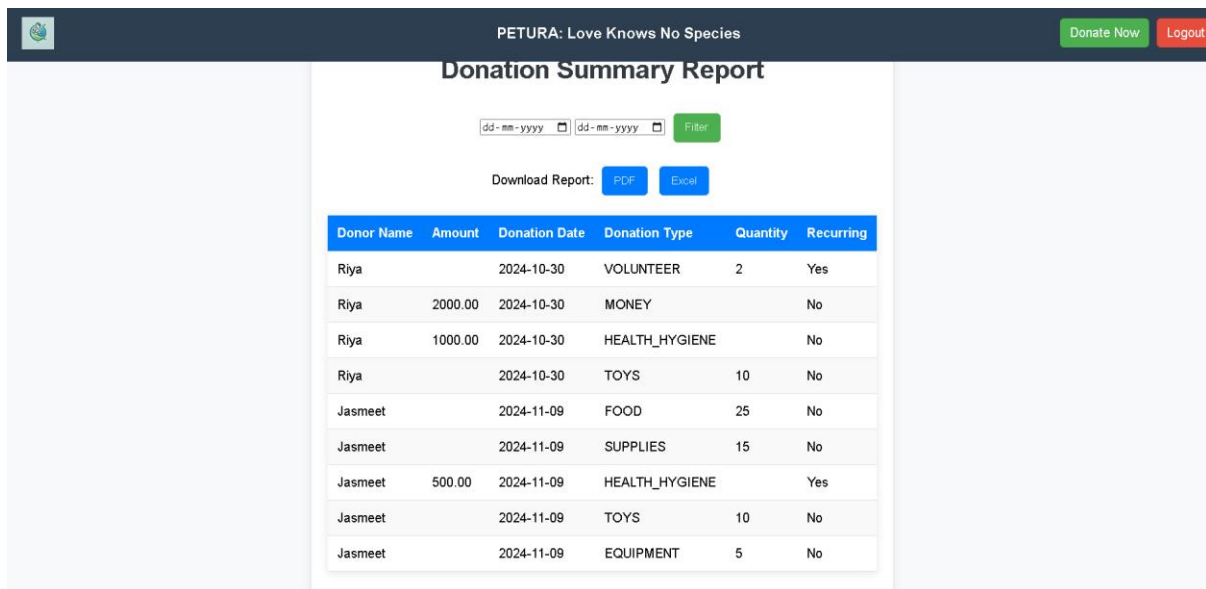
- **Custom Filters:** Both Excel and PDF report generation tools support custom filters to allow users to generate reports based on specific conditions, such as date ranges, animal types, donation categories, or adoption statuses.
- **Date Ranges:** Users can filter the data by time period (e.g., weekly, monthly, yearly) to generate reports that focus on specific timeframes.
- **Categories and Segments:** Reports can be generated by different categories (e.g., by donation type, animal type, or adoption request status) to provide insights into various segments of the system's operations.

4. Exporting and Sharing:

- After the reports are generated, users can download them to their local devices or share them directly via email with other team members, stakeholders, or external organizations.
- For example, admins might send adoption statistics to animal welfare partners, or donation reports to fundraising teams for analysis.

Example Workflow for Report Generation:

1. Select Report Type: The user selects a report type (e.g., Donation Report).
2. Apply Filters: The user sets filters, such as a date range or specific donation type.
3. Generate Report: The user clicks a "Generate Report" button.
4. Download or Share: Once the report is generated, the user can choose to download it as either an Excel or PDF file, or directly share it with others.



Donor Name	Amount	Donation Date	Donation Type	Quantity	Recurring
Riya		2024-10-30	VOLUNTEER	2	Yes
Riya	2000.00	2024-10-30	MONEY		No
Riya	1000.00	2024-10-30	HEALTH_HYGIENE		No
Riya		2024-10-30	TOYS	10	No
Jasmeet		2024-11-09	FOOD	25	No
Jasmeet		2024-11-09	SUPPLIES	15	No
Jasmeet	500.00	2024-11-09	HEALTH_HYGIENE		Yes
Jasmeet		2024-11-09	TOYS	10	No
Jasmeet		2024-11-09	EQUIPMENT	5	No

Fig: 14.3 Report Generation

This functionality provides significant value to the PETURA system by allowing users to generate comprehensive reports in multiple formats, which can be used for internal analysis or external sharing. Whether users need to edit and analyze data in Excel or share polished, professional reports in PDF, this feature supports the system's goal of transparency, data-driven decision-making, and easy distribution of key metrics. if you'd like to expand any section or modify the details

Chapter-15: MAP INTEGRATION

Map integration plays a crucial role in enhancing the functionality of the PETURA animal welfare system, allowing users to interact with geographic data. It provides a visual representation of key locations, such as shelters, incident reports, adoption centers, and animal health incidents. This integration improves the system's usability and helps users make informed decisions about animal welfare based on geographic data.

15.1 Overview of Map Integration (Leaflet)

In PETURA, Leaflet has been chosen as the primary library for map integration due to its lightweight nature, ease of use, and flexibility in rendering interactive maps. Leaflet is an open-source JavaScript library used to build web-based maps. It is well-suited for adding maps to the system and supports various features that enhance user experience, such as:

- **Interactive Mapping:** Users can zoom in and out, pan across different regions, and interact with markers to view more details about specific locations.
- **Customizable Markers:** Leaflet allows for the creation of custom markers that can represent different types of locations, such as shelters, adoption centers, or incident reports.
- **Layer Control:** Multiple layers can be added to a map (e.g., satellite imagery, street view) for more comprehensive map views.
- **Geolocation:** The system can integrate geolocation features that allow users to locate their current position on the map and view nearby shelters or incidents.

This integration makes the system more interactive and user-friendly by providing geographic context to animal-related data.

15.2 Technical Implementation

The technical implementation of the map integration involves the following steps:

1. Integrating Leaflet in PETURA:

- **Frontend (Thymeleaf):** Leaflet's JavaScript library is integrated into the frontend of the PETURA application using Thymeleaf templates. The maps are embedded in specific pages such as shelter locations, incident reporting, and animal management.
- A `<script>` tag is included to load the Leaflet library, and a `<div>` element is created to display the map on the page.

- **Backend (Spring Boot):** The backend (Spring Boot) is responsible for providing the necessary geographic data, such as latitude and longitude coordinates of shelters, animals, or incidents. This data is sent to the frontend via API endpoints, typically in JSON format.

2. Setting Up the Map:

- **Map Initialization:** On page load, the map is initialized using the Leaflet library. The map center and zoom level are configured based on the geographic region of interest.

- **Map Tile Layers:** By default, Leaflet uses OpenStreetMap tiles, but other tile providers (e.g., Google Maps, Mapbox) can be integrated for better visual representation or specialized map views.

```
<!-- Include Leaflet.js -->
<script src="https://unpkg.com/leaflet/dist/leaflet.js"></script>

<script>
  // Initialize the map
  const map = L.map('map').setView([12.912548, 77.690506], 8); // Default location

  // Load OpenStreetMap tiles
  L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
    maxZoom: 19,
    attribution: '© OpenStreetMap'
  }).addTo(map);
```

3. Adding Markers and Popups:

- **Markers** are placed on the map for each relevant location, such as shelters, adoption centers, or locations of reported incidents.

- **Popups** are displayed when a user clicks on a marker. These popups can show detailed information, such as shelter details, adoption status, or incident information.

```
// Add a draggable marker
const marker = L.marker([12.912548, 77.690506], {draggable: true}).addTo(map);

// Update the hidden input field and fetch the human-readable address when the marker is moved
marker.on('moveend', function (e) {
  const position = marker.getLatLng();
  document.getElementById('location').value = position.lat + ', ' + position.lng;

  // Fetch the human-readable address using reverse geocoding
  fetch('https://nominatim.openstreetmap.org/reverse?format=json&lat=${position.lat}&lon=${position.lng}')
    .then(response => response.json())
    .then(data => {
      const address = data.display_name;
      document.getElementById('humanReadableLocation').value = address;
    })
    .catch(error => console.error('Error fetching the address:', error));
});
```

4. Fetching Data from the Backend:

- The frontend makes HTTP requests to the backend to fetch the data required for rendering the map markers (e.g., shelter names, animal locations). This data is fetched through API endpoints and displayed dynamically on the map.

```
@Controller
@RequestMapping("/reports")
public class ReportController {

    @Autowired
    private ReportService reportService;

    @Autowired
    private NotificationService notificationService;

    @Autowired
    private AttachmentRepository attachmentRepository;

    @Autowired
    private AnimalRepository animalRepository;

    @Autowired
    private UserService userService; // Inject UserService for user retrieval

    private static final Logger logger = LoggerFactory.getLogger(ReportController.class);

    // Allow all authenticated users to view the report form
    @PreAuthorize("isAuthenticated()")
    @GetMapping("/new")
    public String showReportForm(Model model) {
        model.addAttribute("report", new Report());
        model.addAttribute("types", ReportType.values()); // Pass ReportType enum values to the form
        return "report-form";
    }

    @PreAuthorize("isAuthenticated()")
    @PostMapping("/submit")
    public String submitReport(@ModelAttribute Report report, @RequestParam("file") MultipartFile[] files,
        Principal principal) {
        // Get the logged-in user by email
        User user = userService.findByEmail(principal.getName());

        if (user == null) {
            // Log and handle case where user is not found
            logger.warn("User not found: {}", principal.getName());
            return "redirect:/error-page"; // Redirect to your error page
        }
    }
}
```

5. Geolocation Integration:

- Geolocation is enabled in the map, allowing users to find their current location. This feature can be useful when users want to find nearby shelters or adoption centers.

```
// Try to get the user's current location using the browser's geolocation API
if (navigator.geolocation) {
  navigator.geolocation.getCurrentPosition(function (position) {
    const userLocation = {
      lat: position.coords.latitude,
      lng: position.coords.longitude,
    };
    map.setView(userLocation, 13); // Zoom in to user's location
    marker.setLatLng(userLocation); // Set the marker to the user's location

    // Fetch the human-readable address for the user's location
    fetch('https://nominatim.openstreetmap.org/reverse?format=json&lat=${userLocation.lat}&lon=${userLocation.lng}')
      .then(response => response.json())
      .then(data => {
        const address = data.display_name;
        document.getElementById('humanReadableLocation').value = address;
      })
      .catch(error => console.error('Error fetching the address:', error));
  }, function (error) {
    console.error('Error getting user location:', error);
  });
} else {
  console.error('Geolocation is not supported by this browser.');
```

6. Incident Reporting Module:

- For the Incident Report Module, users can click on the map to report an incident. The map will capture the coordinates, which are then stored in the system and used to display the location of the reported incident.

15.3 User Interaction with Map

The map integration in PETURA enhances user interaction by providing several features that allow users to engage with the map and view important data points. These features include:

1. **Zoom and Pan:** Users can zoom in and out of the map to view specific locations more clearly and pan across different regions to explore animal shelters, adoption centers, or incident reports in different areas.
2. **Marker Information:** By clicking on the map markers, users can view detailed information about each location. For example, a user can click on a shelter marker to get details about the shelter's capacity, available animals for adoption, and contact information.
3. **Search Functionality:** Users can search for specific locations (e.g., by shelter name, incident type) to quickly locate them on the map. This feature is useful for users who are looking for shelters in a particular region or for specific types of animals.
4. **Geolocation:** Users can use their device's geolocation feature to see their current position on the map, which helps them find the nearest shelters, adoption centers, or incident locations. This is especially useful for users on the move or when reporting incidents.
5. **Interactive Incident Reporting:** Users can click on the map to report incidents, specifying the exact location of the issue. This makes it easier for staff or authorities to respond promptly to the reported incidents.
6. **Map Filtering:** Users can filter the map markers by category (e.g., only show animal shelters, incidents, or adoptions). This helps users focus on the specific data that they are interested in.

7. User-Friendly Interface: The map is designed to be intuitive, with minimalistic controls that allow users to interact with it easily. The markers are clickable, and the popups provide just the right amount of information without overwhelming the user.

Conclusion:

The map integration in PETURA, powered by Leaflet, significantly improves the user experience by providing an interactive, geographic representation of key animal welfare data. Whether users are looking for nearby shelters, viewing reported incidents, or managing animal locations, the map provides a comprehensive, easy-to-navigate solution.

Chapter-16: EMAIL NOTIFICATION AND ALERTS

Email notifications and alerts are integral to keeping users informed and engaged with the PETURA system. Whether it's confirming a registration, acknowledging a donation, tracking the status of an adoption, or alerting users about incidents, these notifications ensure smooth communication and enhance the user experience. This section explains the key email notification functionalities implemented in PETURA.

16.1 Registration Notifications

Upon user registration, PETURA sends an email notification to both the new user and the system administrator. This serves several purposes:

- **New User Confirmation:** A confirmation email is sent to the user after completing the registration process. The email typically includes a welcome message, login instructions, and details about their account.
- **Admin Notification:** Administrators are notified via email about new user registrations. This helps admins keep track of new users and perform any necessary administrative tasks, such as verifying account details.

Features of Registration Notifications:

- Email Subject: "Welcome to PETURA - Registration Confirmation"
- User Email: Includes a personalized greeting, login instructions, and the user's role (e.g., Admin, Staff, or User).
- Admin Email: Admins receive a notification about the new user, including the user's name, email, and registration time, for better management.

```
// Save the user with the selected role
userService.saveUser(userDto);

// Send email to the user
String userSubject = "Welcome to Animal Welfare System";
String userBody = String.format("Dear %s %s,\n\nThank you for registering with us! We are thrilled to welcome you to our community dedicated to animal welfare.\n\n" +
    "If you have any questions or need assistance, feel free to reach out to us.\n\n" +
    "Best regards,\nAnimal Welfare Team",
    userDto.getFirstName(), userDto.getLastName());
notificationService.sendEmail(userDto.getEmail(), userSubject, userBody);

// Fetch admin email using findByRole method
List<User> admins = userService.findByRole("ROLE_ADMIN");
String adminEmail;

// Check if any admins were found
if (!admins.isEmpty()) {
    adminEmail = admins.get(0).getEmail(); // Get the first admin's email
} else {
    adminEmail = "sbp.manmeet@gmail.com"; // Fallback to hardcoded admin email
}

// Send email to the admin
String adminSubject = "New User Registration";
String adminBody = String.format("Dear Admin,\n\nA new user has successfully registered on the platform:\n\n" +
    "Name: %s %s\n" +
    "Email: %s\n" +
    "Role: %s\n\n" +
    "Please review their account and reach out if necessary.\n\n" +
    "Best regards,\nAnimal Welfare System",
    userDto.getFirstName(), userDto.getLastName(), userDto.getEmail(), userDto.getRole());
notificationService.sendEmail(adminEmail, adminSubject, adminBody);
```

Fig: 16.1: User Registration Notification

16.2 Donation Acknowledgements

PETURA automatically sends donation acknowledgment emails to users after a successful donation is made. These notifications express gratitude to donors, provide them with donation details, and sometimes include tax-deductible information if applicable.

Features of Donation Acknowledgements:

- Email Subject: "Thank You for Your Donation!"
- Donor Email: Includes a thank-you message, details of the donation (amount, donor name, date), and a summary of how the donation will be used. If the donation is recurring, users are informed about the next scheduled donation.
- Admin Email: Admins are notified about new donations to update the system or thank the donors personally.

```
@PostMapping("/save")
public String saveDonation(@ModelAttribute Donation donation, RedirectAttributes redirectAttributes) {
    donation.setDate(LocalDate.now()); // Set current date

    try {
        // Save the donation
        donationService.saveDonation(donation);

        // Send confirmation email to the user-provided email
        String userEmail = donation.getEmail();
        String userSubject = "Thank You for Your Donation!";
        String userBody = String.format("Dear %s,\n\n"
            + "Thank you for your generous donation of %s towards our cause. Your support makes a significant difference in the lives of animals in need.\n\n"
            + "If you have any questions or would like to learn more about how your donation is being used, please do not hesitate to reach out.\n\n"
            + "Best regards,\nAnimal Welfare Team", donation.getDonorName(), donation.getQuantity());
        notificationService.sendEmail(userEmail, userSubject, userBody);

        // Fetch admin email using findByRole method
        List<User> admins = userService.findByRole("ROLE_ADMIN");
        String adminEmail;

        // Check if any admins were found
        if (!admins.isEmpty()) {
            adminEmail = admins.get(0).getEmail(); // Get the first admin's email
        } else {
            adminEmail = "sbp.nanmeet@gmail.com"; // Fallback to hardcoded admin email
        }

        // Notify admin of the new donation
        String adminSubject = "New Donation Received";
        String adminBody = String.format("Dear Admin,\n\nA new donation has been successfully recorded:\n\n"
            + "Donor: %s\n" + "Quantity: %s\n" + "Donation Date: %s\n" + "Type: %s\n\n"
            + "Thank you for your attention to this matter.\n\nBest regards,\nAnimal Welfare Team",
            donation.getDonorName(), donation.getQuantity(), donation.getDate(), donation.getDonationType());
        notificationService.sendEmail(adminEmail, adminSubject, adminBody);

        redirectAttributes.addFlashAttribute("message", "Donation saved successfully! Thank you for your contribution.");
        return "redirect:/donations/thank-you"; // Redirect to thank you page
    } catch (IllegalArgumentException e) {
        redirectAttributes.addFlashAttribute("error", e.getMessage());
        return "redirect:/donations"; // Redirect back to the donations page
    }
}
```

Fig: 16.2: Donation Notification

16.3 Adoption Status Tracker

PETURA keeps users informed about the status of their adoption applications. This includes sending email updates when the adoption status changes (e.g., approved, pending, rejected).

Features of Adoption Status Tracker:

- Email Subject: "Update on Your Adoption Application"
- User Email: Includes a message informing the user of the current status of their adoption application (e.g., "Your Adoption Application Has Been Approved"). It may also provide further instructions if any actions are required.

- Admin Email: Admins are notified when an adoption status changes, helping them manage and review adoption requests effectively.

```
// Save the adoption request
adoptionService.createAdoptionRequest(adoptionRequest);

// Send email notification to the user
String userSubject = "Adoption Request Submitted";
String userBody = String.format("Dear %s,\n\nThank you for your interest in adopting! "
    + "We have received your adoption request for the animal '%s' and it is currently pending review."
    + "We appreciate your commitment to providing a loving home."
    + "\n\nOur team will review your request and get back to you shortly. "
    + "If you have any questions in the meantime, please do not hesitate to reach out."
    + "\n\nBest regards,\nAnimal Welfare Team", currentUser.getName(), animal.getName());

notificationService.sendEmail(currentUser.getEmail(), userSubject, userBody);

List<User> admins = userService.findByRole("ROLE_ADMIN");
String adminEmail;

// Check if any admins were found
if (!admins.isEmpty()) {
    adminEmail = admins.get(0).getEmail(); // Get the first admin's email
} else {
    adminEmail = "sbp.manmeet@gmail.com"; // Fallback to hardcoded admin email
}

// Send email notification to the admin
String adminSubject = "New Adoption Request Pending Review";
String adminBody = String.format(
    "Dear Admin,\n\nA new adoption request has been submitted by %s for the animal '%s'. "
    + "Please review the request at your earliest convenience."
    + "\n\nAdoption Request Details:\n- User: %s\n- Animal: %s\n- Status: Pending"
    + "\n\nThank you for your attention to this matter." + "\n\nBest regards,\nAnimal Welfare Team",
    currentUser.getName(), animal.getName(), currentUser.getName(), animal.getName());

notificationService.sendEmail(adminEmail, adminSubject, adminBody);

// Add flash attributes for feedback
redirectAttributes.addFlashAttribute("message",
    "Your adoption request has been submitted and is pending review.");
redirectAttributes.addFlashAttribute("adoptionRequest", adoptionRequest);

// Redirect to the status page
return "redirect:/adoptions/adoption-status";
```

Fig: 16.3: Adoption Notification

16.4 Incident Alerts

Incident alerts are sent to both users and administrators whenever an incident is reported in the system. These alerts ensure timely responses to reported incidents, such as lost animals, emergencies, or any other significant events related to animal welfare.

Features of Incident Alerts:

- Email Subject: "New Incident Reported"
- User Email: Acknowledges the reported incident and provides users with details such as incident type, location, and any next steps if needed.
- Admin Email: Admins receive detailed alerts about the incident report, including location, description, and reporting user, allowing them to respond quickly and allocate resources if necessary.

```

// Email notification logic for admin and staff
String subject = "New Report Submitted: Action Required";
String body = String.format(
    "Dear [Admin/Staff],\n\n" +
    "A new report has been successfully submitted in the system. Kindly review the report and take the necessary actions.\n\n" +
    "Report Details:\n" +
    "- Report ID: %s\n" +
    "- Submitted By: %s\n" +
    "- Report Type: %s\n" +
    "- Status: PENDING\n" +
    "- Description: %s\n\n" +
    "Please log in to the system to review the full report and address any required actions.\n\n" +
    "Thank you for your attention to this matter.\n\n" +
    "Best regards,\n" +
    "The Animal Welfare Team\n" +
    "[Your Organization Name]\n" +
    "[Your Contact Information]",
    savedReport.getId(), user.getName(), savedReport.getType(), savedReport.getDescription());

// Fetch admin email using findByRole method
List<User> admins = userService.findByRole("ROLE_ADMIN");
String adminEmail = admins.isEmpty() ? "sbp.manmeet@gmail.com" : admins.get(0).getEmail(); // Fallback to a default email if none found

// Send the email to the admin
notificationService.sendEmail(adminEmail, subject, body);

// Send the same email to all staff members
/* List<User> staffMembers = userService.findByRole("ROLE_STAFF");
for (User staff : staffMembers) {
    notificationService.sendEmail(staff.getEmail(), subject, body);
}*/

return "redirect:/reports/history"; // Redirect to user's report history
}

```

Fig: 14.4: Incident Alerts

Conclusion

The email notifications and alerts in PETURA are essential in ensuring that users and administrators are kept informed of important events such as new registrations, donations, adoption statuses, and reported incidents. These timely communications help maintain engagement, improve response times, and ensure smooth operation of the system by ensuring all stakeholders are updated on key developments in animal welfare management.

Chapter-17: COMPREHENSIVE VIEW OF APPLICATION

This section provides a detailed view of each module in the PETURA system. For each module, we will discuss the corresponding controller responsible for the functionality and include related HTML pages with screenshots of the page/form output.

17.1 User Module

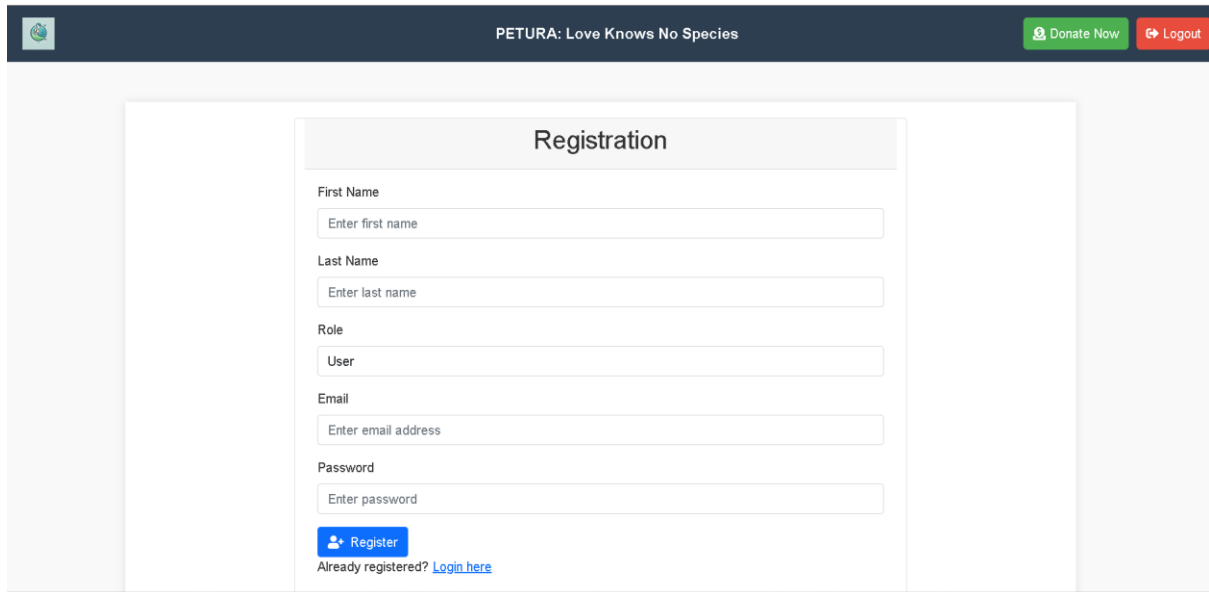
Controller Overview: The `AuthController` is responsible for managing user authentication, registration, and login processes. It handles routes such as login, registration, and redirection based on user roles. It also manages user notifications upon registration and login.

HTML Overview:

1. **index.html:** Likely the landing page or home page.



2. **register.html:** A form for new users to register by entering their details.



The registration form is titled "Registration" and is located within a white container on a dark blue background. The form includes fields for First Name, Last Name, Role (with "User" selected), Email, and Password. A blue "Register" button is at the bottom left, and a link "Already registered? Login here" is at the bottom right. The top navigation bar features the PETURA logo, the site name, and "Donate Now" and "Logout" buttons.

PETURA: Love Knows No Species

Donate Now Logout

Registration

First Name
Enter first name

Last Name
Enter last name

Role
User

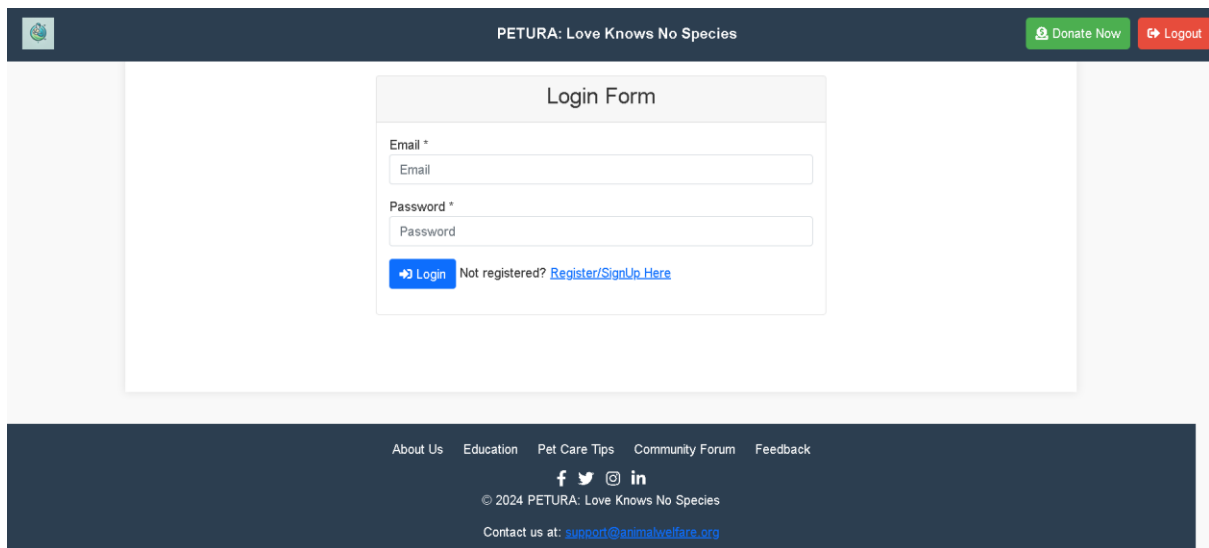
Email
Enter email address

Password
Enter password

Register

Already registered? [Login here](#)

3. **login.html**: A form for users to enter their credentials and authenticate.



The login form is titled "Login Form" and is located within a white container on a dark blue background. It includes fields for Email and Password. A blue "Login" button is at the bottom left, and a link "Not registered? Register/SignUp Here" is at the bottom right. The top navigation bar features the PETURA logo, the site name, and "Donate Now" and "Logout" buttons. The footer contains links for About Us, Education, Pet Care Tips, Community Forum, and Feedback, along with social media icons, copyright information, and contact details.

PETURA: Love Knows No Species

Donate Now Logout

Login Form

Email *
Email

Password *
Password

Login

Not registered? [Register/SignUp Here](#)

About Us Education Pet Care Tips Community Forum Feedback

f t i in

© 2024 PETURA: Love Knows No Species

Contact us at: support@animalwelfare.org

4. **dashboard.html**: Displays role-based information like statistics and quick links after login.

PETURA: Love Knows No Species

Donate Now

Logout

Animal Management

View Animals

Add Animal

Search Animals

Shelter Management

View Shelters

Add Shelter

Search Shelter

Adoption Management

Browse Animals

Review Adoption Request

Check Adoption Status

Welcome to the Animal Welfare Application

Total Animals

9

Total Adoptions

9

Total Incidents

9

Recent Donors

Name	Email	Donation Type	Amount	Quantity	Date
Jasmeet	jas@gmail.com	FOOD		25	2024-11-09
Jasmeet	jas@gmail.com	SUPPLIES		15	2024-11-09
Jasmeet	jas@gmail.com	HEALTH_HYGIENE	500.00		2024-11-09
Jasmeet	jas@gmail.com	TOYS		10	2024-11-09
Jasmeet	jas@gmail.com	EQUIPMENT		5	2024-11-09
Riya	riya@gmail.com	VOLUNTEER		2	2024-10-30
Riya	riya@gmail.com	MONEY	2000.00		2024-10-30

4. **users.html**: A page for managing users, managed by admin.

PETURA: Love Knows No Species

Donate Now

Logout

Registered Users

First Name	Last Name	Email
manmeet	chhabra	sbp.manmeet@gmail.com
riya	kaur	riya@gmail.com
nag	raj	nag@gmail.com
nag	raj	nag1@gmail.com
abc	xyz	abc@gmail.com
Gagandeep	Chhabra	gagan.singh2092@gmail.com
asdf	scsdf	as@gmail.com
Jas	meet	jas@gmail.com

About Us

Education

Pet Care Tips

Community Forum

Feedback

17.2 Animal Module

Controller Overview: The `AnimalController` manages all animal-related operations, including adding new animals, updating, deleting, and listing animals. It also handles animal searches and notifications to admins when animals are created, updated, or deleted.

HTML Overview:

1. **animal-list.html**: Displays a list of animals available for adoption.



PETURA: Love Knows No Species

[Donate Now](#) [Logout](#)

Animal List

Filter by Type:

All

Filter

ID	Name	Type	Actions
3	xyz	cat	Details Edit Delete
4	Dog	Unknown	Details Edit Delete
5	cat	Unknown	Details Edit Delete
6	Cow	Unknown	Details Edit Delete
7	Unknown	Unknown	Details Edit Delete
12	Fluffy	Dog	Details Edit Delete
16	gfc	fxdg	Details Edit Delete

[About Us](#) [Education](#) [Pet Care Tips](#) [Community Forum](#) [Feedback](#)

© 2024 PETURA: Love Knows No Species

Contact us at: support@animalwelfare.org

2. **animal-details.html**: Shows detailed information about a specific animal.

3. **animal-create.html**: Form for admins or staff to add new animals to the system.

PETURA: Love Knows No Species		
		Donate Now Logout
Create Animal		
Name: Enter animal name		
Type: Select Type		
Health Status: Healthy		
Doctor Appointment: Select		
Shelter: Happy Paws Shelter		
Adoption Status: Available		
Picture: Choose File No file chosen		
Create Animal		

4. **animal-search.html**: A search page for filtering animals.

5. **animal-shelter-list.html**: A list page for displaying animals by shelter.

6. **animal-edit.html**: Form for admins or staff for editing animal details.

PETURA: Love Knows No Species [Donate Now](#) [Logout](#)

Type:
Dog

Health Status:
Healthy

Doctor Appointment:
No

Picture Data:
Choose File No file chosen

Shelter:
Happy Paws Shelter

[Update Animal](#)

[Back to Animal List](#)

About Us Education Pet Care Tips Community Forum Feedback

f t i in

© 2024 PETURA: Love Knows No Species

17.3 Shelter Module

Controller Overview: The `ShelterController` handles shelter-related actions such as listing shelters, adding shelter details, and viewing shelter capacities. It also facilitates the management of shelter staff.

HTML Overview:

1. **shelter-list.html:** Lists all shelters with basic information.

PETURA: Love Knows No Species [Donate Now](#) [Logout](#)

Shelter List

Select Name Select Location Capacity Search

Name	Location	Capacity	Current Occupancy	Contact Details	Actions
Happy Paws Shelter	123 Dog Lane	150	N/A		Manage Capacity Of Shelter View Animals Of The Shelter Shelter Details Edit Shelter Details Add Staff To The Shelter Delete Shelter
Furry Friends Shelter	456 Cat Ave	30	N/A		Manage Capacity Of Shelter View Animals Of The Shelter Shelter Details Edit Shelter Details Add Staff To The Shelter Delete Shelter

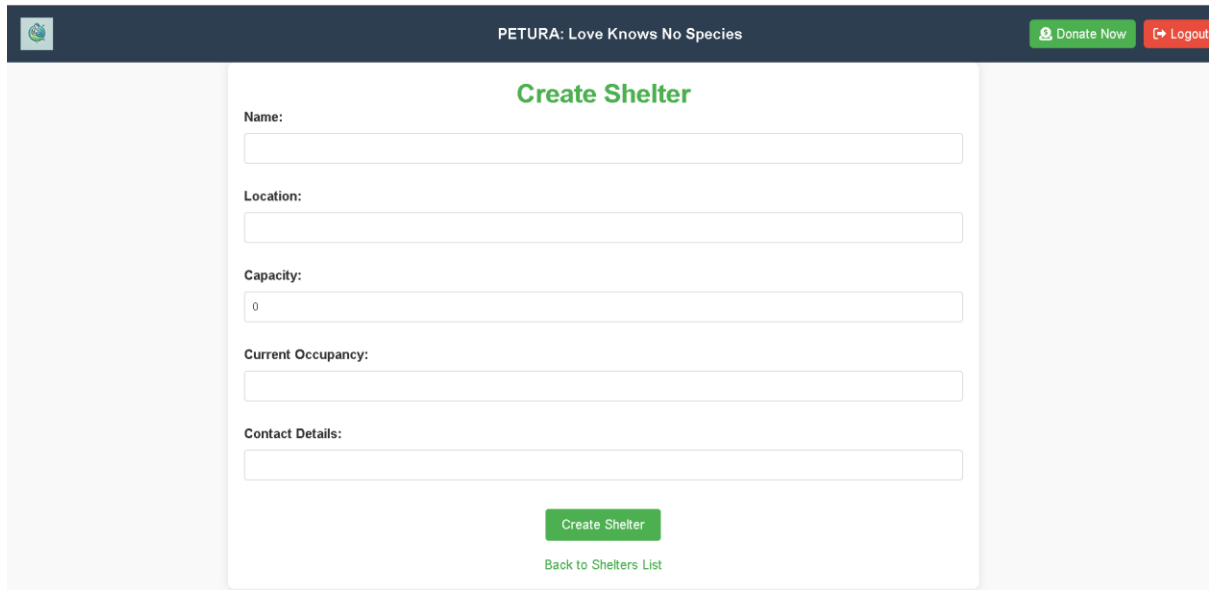
23°C Haze

Search

ENG IN 21:36 11-11-2024

2. **shelter-details.html:** Displays detailed information about a specific shelter.

3. **shelter-create.html:** A form to create and add new shelters.

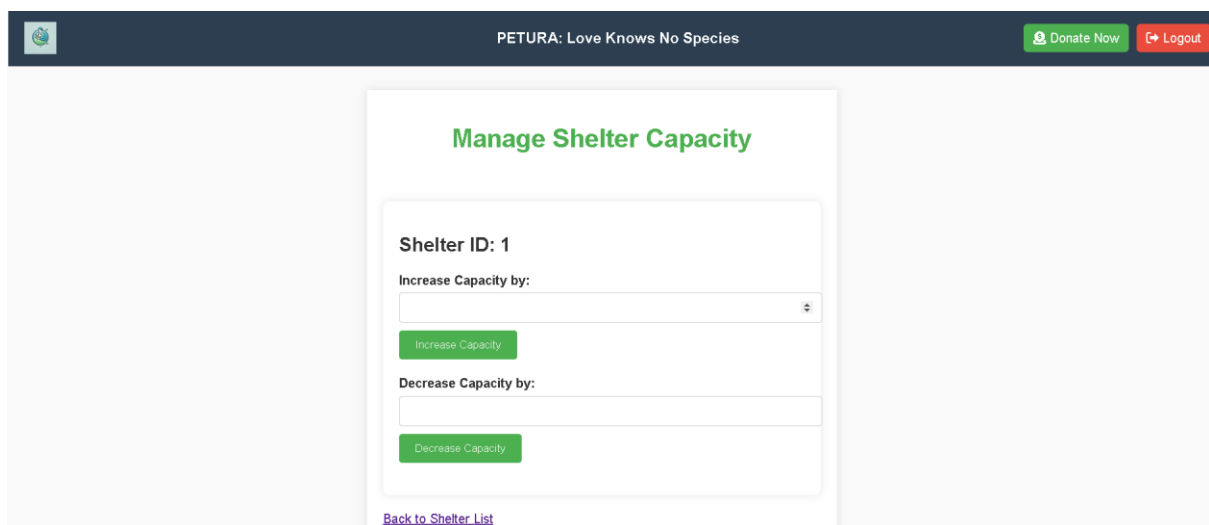


The screenshot shows the 'Create Shelter' form in the PETURA application. The form is titled 'Create Shelter' in green. It contains five input fields: 'Name:', 'Location:', 'Capacity:' (with a value of 0), 'Current Occupancy:', and 'Contact Details:'. Below the form is a green 'Create Shelter' button and a green link 'Back to Shelters List'. The header of the application is dark blue with the text 'PETURA: Love Knows No Species' and two buttons: 'Donate Now' and 'Logout'.

4. **shelter-edit.html**: A page for editing shelter details.

5. **shelter-search.html**: A search page for filtering shelters.

6. **shelter-capacity.html**: Page showing the shelter's capacity, probably for admins.



The screenshot shows the 'Manage Shelter Capacity' form in the PETURA application. The form is titled 'Manage Shelter Capacity' in green. It contains a 'Shelter ID: 1' label, an 'Increase Capacity by:' input field with a green 'Increase Capacity' button, and a 'Decrease Capacity by:' input field with a green 'Decrease Capacity' button. Below the form is a green link 'Back to Shelter List'. The header of the application is dark blue with the text 'PETURA: Love Knows No Species' and two buttons: 'Donate Now' and 'Logout'.

7. **shelter-staff-add.html**: Page to add staff to a specific shelter..

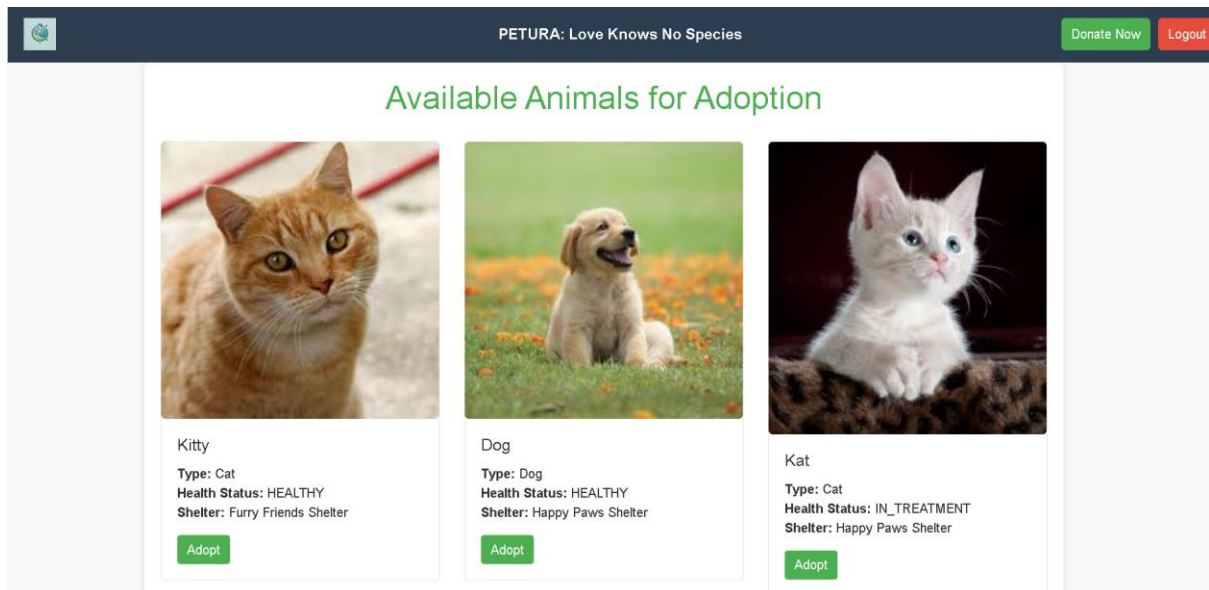
8. **shelter-staff-list.html**: Page to display list of staff available.

17.4 Adoption Module

Controller Overview: The `AdoptionController` manages the adoption process. This includes listing available animals for adoption, handling adoption requests, processing eligibility questionnaires, and managing the review of requests by admins.

HTML Overview:

1. **adopt-animal-list.html**: List of animals available for adoption.



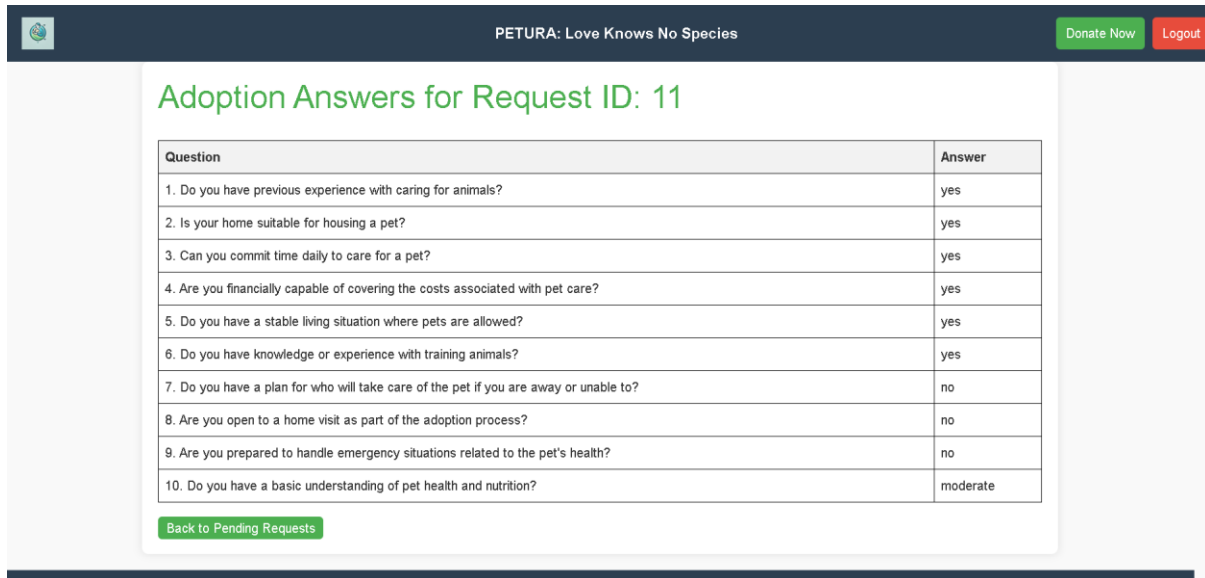
2. **adoption-status.html**: Displays the current status of adoption requests for users.

3. **adoption-request-review.html**: A page where admins review and approve/reject adoption requests.



4. **adoption-request-form.html**: A form that potential adopters fill out to express interest in adopting an animal.

5. **adoption-answers.html**: A page to display adoption answers (likely a review page for admins).



Question	Answer
1. Do you have previous experience with caring for animals?	yes
2. Is your home suitable for housing a pet?	yes
3. Can you commit time daily to care for a pet?	yes
4. Are you financially capable of covering the costs associated with pet care?	yes
5. Do you have a stable living situation where pets are allowed?	yes
6. Do you have knowledge or experience with training animals?	yes
7. Do you have a plan for who will take care of the pet if you are away or unable to?	no
8. Are you open to a home visit as part of the adoption process?	no
9. Are you prepared to handle emergency situations related to the pet's health?	no
10. Do you have a basic understanding of pet health and nutrition?	moderate

Back to Pending Requests

6. **adoption-questionnaire.html**: Adoption questionnaire form.

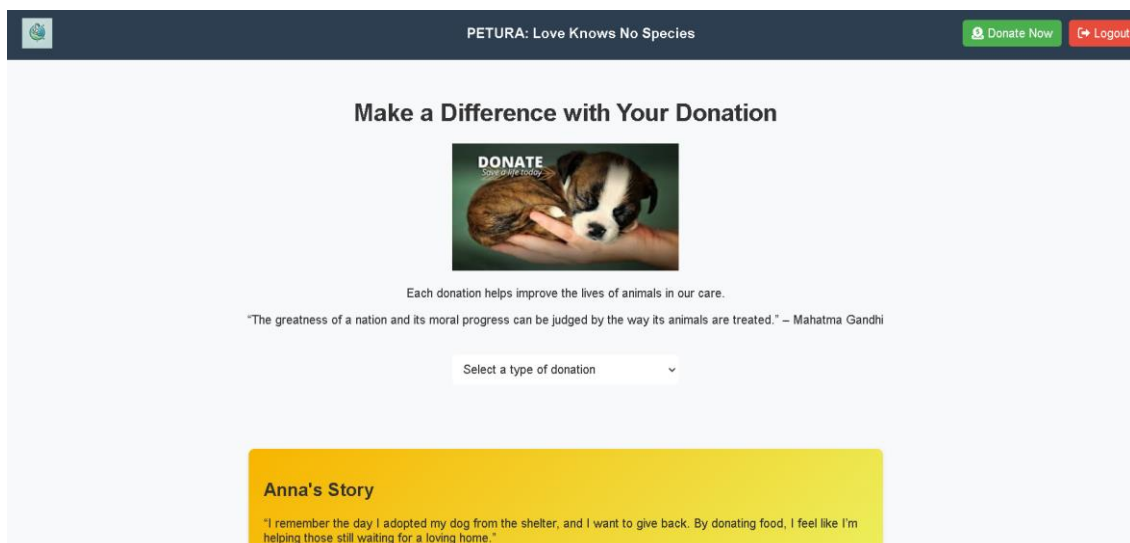
7. **adoption-result.html**: Page showing the result of an adoption request.

17.5 Donation Module

Controller Overview: The `DonationController` handles donation-related activities, such as viewing donation history, creating new donations, and displaying the list of recent donations. It also manages different donation types (monetary, food, equipment, etc.).

HTML Overview:

1. **main-donation-page.html**: Likely a general donation page with various donation options.



Make a Difference with Your Donation

DONATE
Save a life today

Each donation helps improve the lives of animals in our care.

"The greatness of a nation and its moral progress can be judged by the way its animals are treated." – Mahatma Gandhi

Select a type of donation

Anna's Story

"I remember the day I adopted my dog from the shelter, and I want to give back. By donating food, I feel like I'm helping those still waiting for a loving home."

- 2. **donor-history.html**: Displays the donation history for logged-in users.
- 3. **recent-donors.html**: A scrolling list that shows recent donors.



PETURA: Love Knows No Species

Donate Now

Logout

Recent Donors

Donation Type	Donor Name	Quantity	Amount	Date
Food	Jasmeet	25	N/A	2024-11-09
Supplies	Jasmeet	15	N/A	2024-11-09
Health and Hygiene Products	Jasmeet	N/A	\$ 500.00	2024-11-09
Toys and Enrichment	Jasmeet	10	N/A	2024-11-09
Equipment	Jasmeet	5	N/A	2024-11-09
Volunteer Time	Riya	2	N/A	2024-10-30
Monetary Donations	Riya	N/A	\$ 2000.00	2024-10-30
Health and Hygiene Products	Riya	N/A	\$ 1000.00	2024-10-30
Toys and Enrichment	Riya	10	N/A	2024-10-30

About UsEducationPet Care TipsCommunity ForumFeedback

f

t

@

in

- 4. **donation-list.html**: List page for displaying donations.
- 5. **thank-you.html**: Thank you page after a donation or other successful action.
- 6. **doantion-equipment.html**:



PETURA: Love Knows No Species

Donate Now

Logout



Help Animals Thrive with Essential Equipment

When young animals, like Sparky the Kitten, are brought to our shelter, they often arrive scared and in need of gentle care. Proper equipment helps us create safe, comfortable spaces where they can recover and grow. Thanks to donors like you, we've been able to provide warm bedding, soft carriers, and the training tools essential for these animals to become happy, healthy, and ready for new homes. Every piece of equipment you donate supports the animals on their journey, offering them not only safety but a second chance at life.

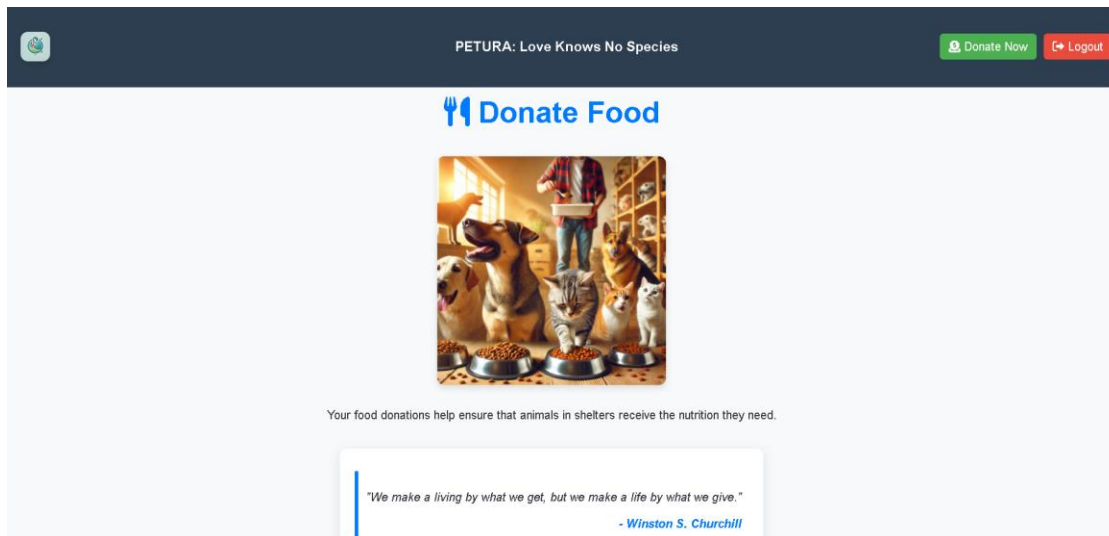
Quantity:

Quantity

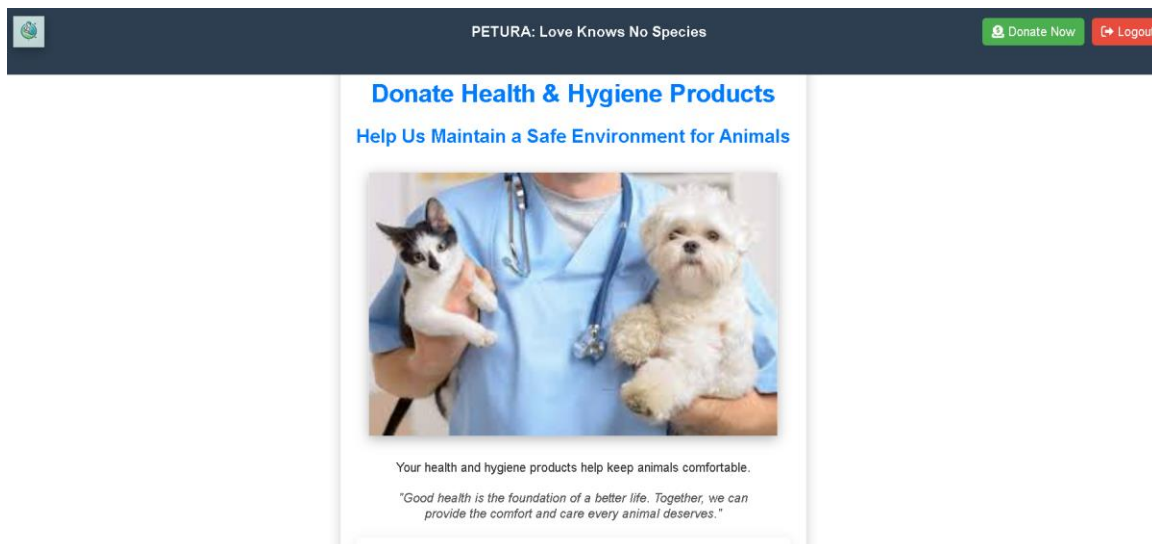
Your Name:

Your Name

- 7. **donation-educatinal.html**:
- 8. **donation-food.html**:



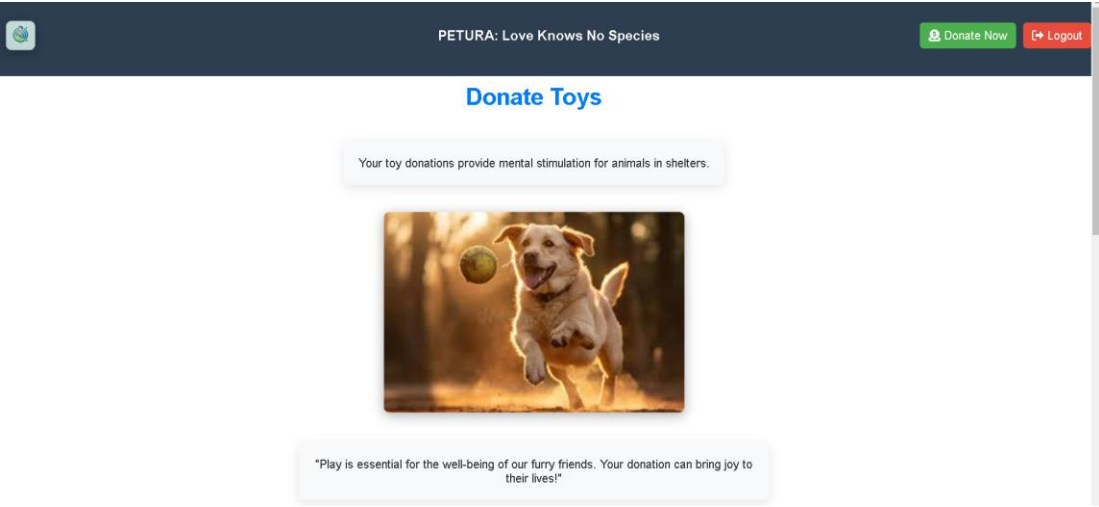
9. donation-health-hygiene.html:



10. donation-money.html:

11. donation-supplies.html:

12. donation-toys.html:



13. donation-volunteer.html:

17.6 Report Generation Module

Controller Overview: The `GenerateReportController` is responsible for generating custom reports related to animals, adoptions, donations, and more. It supports the generation of reports in both Excel and PDF formats.

HTML Overview:

1. adoption-trends.html:

PETURA: Love Knows No Species

Donate Now

Logout

Start Date:

End Date:

Filter

Adoption trends retrieved successfully.

Download Report:

PDF

Excel

Animal Name	Adopter Name	Adoption Date	Status	Request Date	Score
Kitty	riya kaur		REJECTED	2024-10-20	54
Kitty	riya kaur		REJECTED	2024-10-20	54
Kitty	riya kaur		REJECTED	2024-10-20	54
Kitty	riya kaur		REJECTED	2024-10-20	18
Kitty	riya kaur		REJECTED	2024-10-20	54
Kitty	riya kaur		PENDING	2024-10-20	63
Fluffy	manmeet chhabra		PENDING	2024-11-09	46
Fluffy	riya kaur		APPROVED	2024-11-09	82
Fluffy	riya kaur		REJECTED	2024-11-09	0

2. animal-distribution.html:

3. animal-health-status.html:

4. animal-reports:.html:

PETURA: Love Knows No Species					
Download Report: PDF Excel					
Animal Name	Type	Health Status	Doctor Appointment	Shelter Name	Adoption Status
Kitty	Cat	HEALTHY		Furry Friends Shelter	
Dog	Dog	HEALTHY	Yes	Happy Paws Shelter	
Kat	Cat	IN_TREATMENT	Yes	Happy Paws Shelter	
Cow	Cow	HEALTHY	Yes	Happy Paws Shelter	
Unknown	Unknown	Unknown	No Appointment	No Shelter Assigned	
Fluffy	Dog	HEALTHY	No	Happy Paws Shelter	
gfc	fxdg	HEALTHY	No	Happy Paws Shelter	
Unknown	Unknown	Unknown	No Appointment	No Shelter Assigned	
Unknown	Unknown	Unknown	No Appointment	No Shelter Assigned	
About Us Education Pet Care Tips Community Forum Feedback					
© 2024 PETURA: Love Knows No Species					
Contact us at: support@peturaindia.org					

5. donation-summary.html:

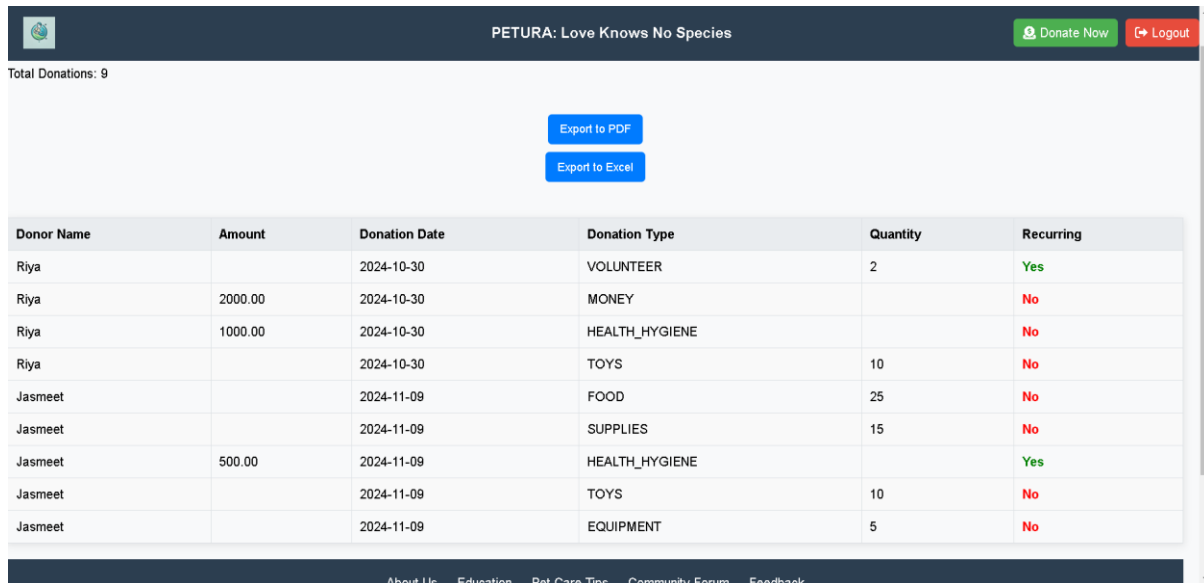
6. incident-reports.html:

PETURA: Love Knows No Species						
Start Date: dd - mm - yyyy End Date: dd - mm - yyyy Filter Export to PDF Export to Excel						
Report Type	Status	Description	Report Date	Location	Address	Rejection Reason
ACCIDENT	rejected	Animal accident	2024-10-21 00:44	12.912548477916772,77.69050598144533		xc
ABUSE	Rescued	animal abuse	2024-10-21 00:48	12.913719718332521,77.68681526184083		dfg
ACCIDENT	approved	Animal accident	2024-10-21 01:10	12.913175928822163,77.68818852429666	Sunny Brooks, Kaikondrahalli, Doddakannalli, Bangalore East, Bengaluru Urban, Karnataka, 560035, India	N/A
ABUSE	rejected	animal	2024-10-21 01:24	12.91334324879754,77.68801688576755	Sunny Brooks, Kaikondrahalli, Doddakannalli, Bangalore East, Bengaluru Urban, Karnataka, 560035, India	dcfg
ABUSE		asdfg	2024-10-21 02:24	12.91104258931668,77.68363968630194	Sarjapura Road, Kaikondrahalli, Bengaluru, Bangalore East, Bengaluru Urban, Karnataka, 560035, India	N/A
ABUSE		fdfgh	2024-10-21 02:24	12.91456231576855,77.68406540928400	Greenwood Regency, Kaikondrahalli, Bangalore East, Bengaluru Urban, Karnataka, 560035, India	N/A

7. shelter-capacity.html:

8. total-adoptions.html:

9. total-donations.html:



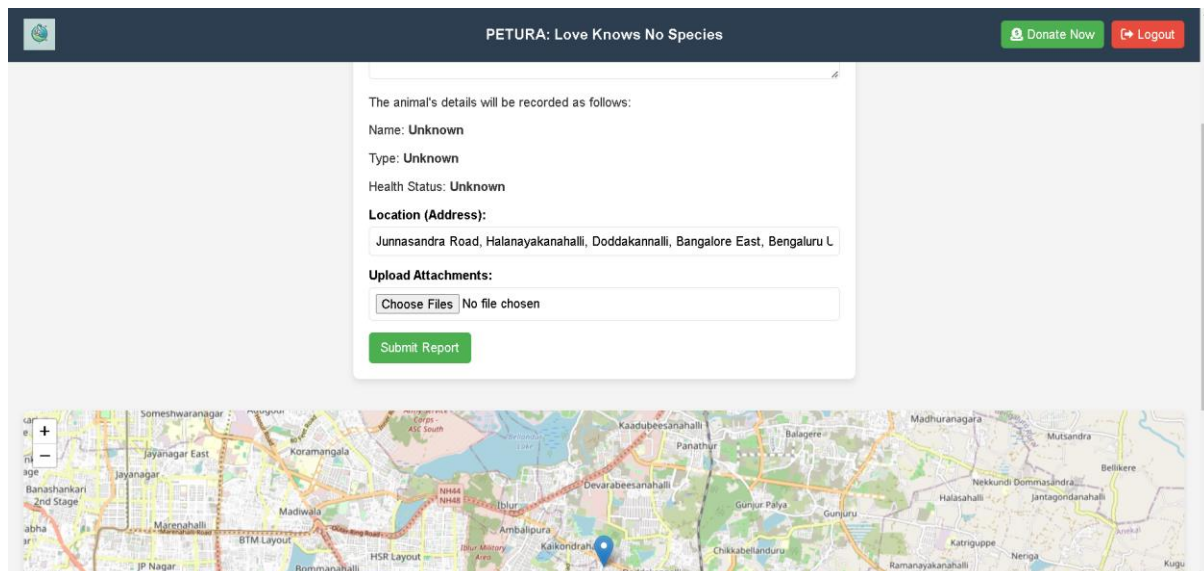
Donor Name	Amount	Donation Date	Donation Type	Quantity	Recurring
Riya		2024-10-30	VOLUNTEER	2	Yes
Riya	2000.00	2024-10-30	MONEY		No
Riya	1000.00	2024-10-30	HEALTH_HYGIENE		No
Riya		2024-10-30	TOYS	10	No
Jasmeet		2024-11-09	FOOD	25	No
Jasmeet		2024-11-09	SUPPLIES	15	No
Jasmeet	500.00	2024-11-09	HEALTH_HYGIENE		Yes
Jasmeet		2024-11-09	TOYS	10	No
Jasmeet		2024-11-09	EQUIPMENT	5	No

17.7 Incident Report Module

Controller Overview: The `ReportController` handles incident reporting related to animal welfare. Users can report incidents related to animals, shelters, or adoption issues. The controller processes these reports and displays them to admins for review.

HTML Overview:

1. **report-form.html:** A form to submit new incident reports regarding animal welfare.



The animal's details will be recorded as follows:

Name: Unknown

Type: Unknown

Health Status: Unknown

Location (Address):

Junnasandra Road, Halanayakanahalli, Doddakannalli, Bangalore East, Bengaluru L

Upload Attachments:

Choose Files No file chosen

Submit Report

2. **incident-reports.html:** Displays a list of reported incidents, categorized by their status.

3. **report-history.html:** Displays the report history of the user.

4. **admin-reports.html:** Display list of incident reports.

PETURA: Love Knows No Species				
			Donate Now	Logout
All Reports				
Description All Types All Statuses Search				
ID	Type	Description	Status	Actions
3	ACCIDENT	Animal accident	rejected	View Details Reject Approve
4	ABUSE	animal abuse	Rescued	View Details Reject Approve
5	ACCIDENT	Animal accident	approved	View Details Reject Approve
6	ABUSE	animal	rejected	View Details Reject Approve
7	ABUSE	asdfg		View Details Reject Approve
8	ABUSE	fvgbh		View Details Reject Approve
10	ABUSE	vbnm		View Details Reject Approve
11	ACCIDENT	Cat got in an accident	rejected	View Details Reject Approve

5. **admin-reports-pending:** Display list of pending incident reports.

PETURA: Love Knows No Species				
			Donate Now	Logout
Pending Reports				
ID	Type	Description	Status	Actions
12	ACCIDENT	Cat got in an accident	PENDING	View Details Reject Report Approve Report

[About Us](#)
[Education](#)
[Pet Care Tips](#)
[Community Forum](#)
[Feedback](#)

© 2024 PETURA: Love Knows No Species
 Contact us at: support@animalwelfare.org

Chapter-18: CHALLENGES FACED

During the development of PETURA, several challenges emerged. These were overcome through research, collaboration, and iterative development. Here are the primary challenges:

18.1 Data Validation

- **Challenge:** Ensuring accurate data entry was crucial, especially for user information, animal records, adoption requests, and donations.
- **Solution:** Robust validation mechanisms were implemented at both the frontend and backend. Specific validation checks were added for fields such as email, phone numbers, and required fields to avoid errors and maintain data integrity.

18.2 Notification Management

- **Challenge:** Managing and scheduling notifications (emails) for various events, such as user registration, adoption updates, and donation confirmations, proved complex, particularly with high volumes of users and actions.
- **Solution:** A notification service was developed to queue and batch notifications efficiently. JavaMail and Spring Scheduler handled immediate and recurring notifications, ensuring timely communication without overloading the system.

18.3 Performance Scalability

- **Challenge:** The application needed to handle a growing number of users, animals, and shelter records, along with extensive data filtering and search capabilities, without degrading performance.
- **Solution:** Performance optimizations were made in query design, caching strategies, and indexing database tables. These adjustments helped ensure that the application could scale to support a larger user base while maintaining quick response times.

18.4 Security Concerns

- **Challenge:** Security was paramount, as PETURA handles sensitive data, including user information, payment data for donations, and adoption details.
- **Solution:** Security measures included encrypting sensitive data, implementing role-based access controls, and applying regular vulnerability checks. Input validation, password hashing, and session management further strengthened the application against potential attacks.

Chapter-19: FUTURE ENHANCEMENTS

PETURA has the potential for further improvements and new features to better serve users and enhance the platform's functionality. Proposed future enhancements include:

1. **Mobile Application**: Develop a mobile app to improve accessibility and user experience on mobile devices.
2. **Enhanced AI-Based Animal Matching**: Integrate AI algorithms to improve matching potential adopters with suitable animals based on user preferences and animal profiles.
3. **Automated Report Generation**: Expand reporting to include customizable, automated reports on adoption trends, donation statistics, and animal intake metrics.
4. **Multi-Language Support**: Provide language options to increase accessibility for a diverse audience.

Chapter-20: CONCLUSION

PETURA: Love Knows No Species is a purpose-driven animal welfare management system designed to support the essential functions of animal shelters. By integrating user registration, animal management, adoption processing, donation tracking, and incident reporting into a single platform, PETURA streamlines shelter operations, enabling staff to focus on animal care and adoption.

The system's automated notifications enhance communication, keeping users, staff, and donors informed of key updates and actions, while built-in security and data validation ensure privacy and data integrity. PETURA's reporting features also support data-driven insights, allowing shelters to make informed decisions and improve their services.

PETURA has been designed with scalability and flexibility in mind, positioning it to meet the needs of shelters of all sizes. Looking ahead, potential enhancements like a mobile app, AI-powered animal matching, and multilingual support offer exciting growth opportunities.

Ultimately, PETURA is more than a management tool—it's a community-focused platform that fosters efficient shelter operations, responsible adoption, and compassionate animal care. Through this system, PETURA is helping shelters connect more animals with loving homes, making a meaningful impact in the world of animal welfare.

The full source code is available on GitHub for reference, or customization: [\[GitHub Link\]](#).

Chapter-21: REFERENCES

A list of resources and references used in the development of PETURA, including technical documentation, libraries, and frameworks:

- Spring Boot Documentation. Available at: <https://spring.io/projects/spring-boot>
- Thymeleaf Documentation. Available at: <https://www.thymeleaf.org/documentation.html>
- JavaMail API Reference. Available at: <https://javaee.github.io/javamail/>
- Hibernate Documentation. Available at: <https://hibernate.org/orm/documentation/>
- Bootstrap Documentation. Available at: <https://getbootstrap.com/docs/>
- PETURA Source Code on GitHub. Available at: [\[GitHub Link\]](#)

Chapter-22: APPENDIX

The Appendix section includes additional information supporting PETURA, such as screenshots, or code snippets.

- Appendix A: Application Properties

```
1 spring.application.name=demo
2
3 spring.datasource.url=jdbc:mysql://localhost:3306/animalwelfaresyst
4 spring.datasource.username=root
5 spring.datasource.password=password
6
7 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
8 spring.jpa.hibernate.ddl-auto=update
9
10 spring.jpa.show-sql=true
11 spring.jpa.properties.hibernate.format_sql=true
12
13
14 logging.level.org.springframework.web=DEBUG
15 logging.level.org.springframework.boot.autoconfigure.web=DEBUG
16
17
18 spring.mail.host=smtp.gmail.com
19 spring.mail.port=587
20 spring.mail.username=manmeetshabha04@gmail.com
21 spring.mail.password=crfvdvvsybpddn
22 spring.mail.properties.mail.smtp.auth=true
23 spring.mail.properties.mail.smtp.starttls.enable=true
24
25
26 spring.servlet.multipart.enabled=true
27 spring.servlet.multipart.max-file-size=10MB
28 spring.servlet.multipart.max-request-size=10MB
29
```

- Appendix B: Package Structure

- ▼  AnimalWelfareSystem4 [boot] [workspace-spring-tool-suite-4-4.24.0.RELEASE-1 final-application]
 - ▼  src/main/java
 - >  com.manmeet.animalsys
 - >  com.manmeet.animalsys.config
 - >  com.manmeet.animalsys.controller
 - >  com.manmeet.animalsys.dto
 - >  com.manmeet.animalsys.entity
 - >  com.manmeet.animalsys.repos
 - >  com.manmeet.animalsys.security
 - >  com.manmeet.animalsys.service
 - >  com.manmeet.animalsys.service.impl
 - >  com.manmeet.animalsys.utility
 - >  src/main/resources
 - >  src/test/java
 - >  JRE System Library [JavaSE-22]
 - >  Maven Dependencies
 - >  target/generated-sources/annotations
 - >  target/generated-test-sources/test-annotations
 - >  src
 - >  target
 - >  HELP.md
 - >  mvnw
 - >  mvnw.cmd
 - >  pom.xml

- Controller Classes

- ▼  com.manmeet.animalsys.controller
 - >  AdoptionController.java
 - >  AnimalController.java
 - >  AuthController.java
 - >  DonationController.java
 - >  GenerateReportController.java
 - >  PageController.java
 - >  ReportController.java
 - >  ShelterController.java

- DTO Classes

- ▼  com.manmeet.animalsys.dto
 - >  AdoptionReportDto.java
 - >  AnimalCountDto.java
 - >  AnimalReportDto.java
 - >  DonationReportDto.java
 - >  ReportSummaryDto.java
 - >  ShelterCapacityDto.java
 - >  UserDto.java

- Entity Classes

- ▼  com.manmeet.animalsys.entity
 - >  Adoption.java
 - >  AdoptionAnswer.java
 - >  AdoptionRequestStatus.java
 - >  AdoptionStatus.java
 - >  Animal.java
 - >  Attachment.java
 - >  Donation.java
 - >  DonationType.java
 - >  Report.java
 - >  ReportType.java
 - >  Role.java
 - >  Shelter.java
 - >  User.java

- Repository Classes

- ▼  com.manmeet.animalsys.repos
 - >  AdoptionAnswerRepository.java
 - >  AdoptionRepository.java
 - >  AnimalRepository.java
 - >  AttachmentRepository.java
 - >  DonationRepository.java
 - >  ReportRepository.java
 - >  RoleRepository.java
 - >  ShelterRepository.java
 - >  UserRepository.java

- Service Interface and it's Implementations

- ▼  com.manmeet.animalsys.service
 - >  AdoptionService.java
 - >  AnimalService.java
 - >  DonationScheduler.java
 - >  DonationService.java
 - >  GenerateReportService.java
 - >  NotificationService.java
 - >  ReportService.java
 - >  ShelterService.java
 - >  UserService.java
- ▼  com.manmeet.animalsys.service.impl
 - >  AdoptionServiceImpl.java
 - >  AnimalServiceImpl.java
 - >  DonationSchedulerImpl.java
 - >  DonationServiceImpl.java
 - >  GenerateReportServiceImpl.java
 - >  NotificationServiceImpl.java
 - >  ReportServiceImpl.java
 - >  ShelterServiceImpl.java
 - >  UserServiceImpl.java

- Utility Classes

- ▼  com.manmeet.animalsys.utility
 - >  AnimalSpecification.java
 - >  EmailUtility.java
 - >  ReportUtil.java
-

Chapter-23: Detailed Plan of Work for 16 Weeks

The following table summarizes the detail plan of work against the planned duration in weeks.

Serial No.	Tasks or Subtasks	Planned Duration (Weeks)	Specific Deliverable	Status
1	Requirement Gathering & Analysis	1 st and 2 nd Week	The detailed requirements as agreed upon by the users and supervisor.	Completed
2	Setting up Eclipse IDE, SQL server and configuration Setup	3 rd and 4 th Week	Installing Eclipse, MS-SQL, Tomcat Server, Maven, Postman etc.	Completed
3	Creating UI Design	5 th and 6 th Week	Creating the UI pages for the application	Completed
Mid-Semester Report Submission				
4	Developing the Services	7 th , 8 th , 9 th and 10 th Week	Writing code using Java, Spring boot and integrating the services.	Completed
5.	Creating 1 Database Table	11 th Week	Creating Database and SQL Queries to store data and for fetching the data from DB	Completed
5	Debugging and testing the application	12 th , 13 th and 14 th Week	End to end testing of the Application and debugging	Completed
6	Finalize the document	15 th Week	Get the document reviewed from the Supervisor	Completed
7	Final Demo on the Application	16 th Week	Give the demo to the supervisor and note down the feedback if any.	Completed
Final Report Submission				